

Optimization of Multilevel Feedback Queue Scheduling and Queueing Models Using Simulation and Analytical Techniques

ENDRI BAKU AND HAZIS VODA

Epoka University, Faculty of Architecture and Engineering, Tirana, Albania
(email: ebaku23@epoka.edu.al, hvoda23@epoka.edu.al)

SUMMARY

The default time-quantum values used by Round-Robin schedulers in multilevel feedback queue (MLFQ) systems are almost never optimal for a given workload. This paper poses the tuning of a three-level MLFQ as an empirical optimisation problem and pairs the result with closed-form queueing theory. A discrete-event MLFQ simulator was implemented in Python and run under a Monte Carlo grid search over the time quanta $Q1$ and $Q2$, the duty-cycle allotments $L1$ and $L2$, and — in a second variant — the SJF/FCFS split fraction T at the bottom level. Across 256 valid configurations and 25,600 simulation runs we identify the optimal $(Q1, Q2, L1, L2[, T])$ per objective. Adding a Shortest-Job-First sub-phase to the bottom level cuts the mean turnaround time by 48% (from 168,694 to 87,800 time units) versus the FCFS-only baseline; a small $Q1$ minimises response time while a large $Q1$ minimises turnaround — the classic Round-Robin quantum trade-off in quantified form. We then compute M/M/1 and M/M/S queueing quantities ($\rho, L, L_q, W, W_q, P_n$) analytically and cross-check them against a discrete-event simulator; mean sojourn time agrees with the closed form to within 0.4% at 60,000 arrivals. A randomness study under three inter-arrival distributions shows that distribution *shape* alone moves mean queue wait by a factor of roughly three even at identical mean rates.

KEY WORDS: multilevel feedback queue, round-robin scheduling, time quantum, shortest-job-first, Monte Carlo, M/M/1, M/M/S, queueing theory, randomness study, discrete-event simulation

1. Introduction

CPU scheduling determines, more than almost any other operating-system decision, what a user actually experiences as system performance. A scheduler that picks the wrong job at the wrong moment turns a system with plenty of compute capacity into one that feels unresponsive, even when total throughput is healthy [1,2]. Modern general-purpose operating systems rarely use a single scheduling policy: they layer policies in a multilevel feedback queue (MLFQ), in which interactive jobs are kept in a high-priority queue serviced by Round-Robin (RR) with a small time quantum, while CPU-bound jobs sink to lower-priority queues where the quantum is larger or where First-Come-First-Served (FCFS) takes over entirely.

The textbook MLFQ has at least four knobs that the operating-system designer (or tuner) must pick: the RR quantum $Q1$ at the top level, the RR quantum $Q2$ at the middle level, and the time allotted to each level — $L1$, $L2$, and a derived $L3$. With Shortest-Job-First introduced as a bottom-level alternative, a fifth knob T appears: the fraction of $L3$ budget given to SJF before FCFS takes over. None of these knobs has an analytic optimum; their effect on turnaround, waiting and response time is workload-dependent and depends on at least the *distribution* of process arrival times and burst durations [2]. The default values printed in textbooks (often $Q1 = 10$ ms, $Q2 = 100$ ms) are reasonable starting points but not optimal.

This paper takes that observation seriously and treats MLFQ tuning as a *black-box optimisation problem*. Concretely we contribute:

1. A reproducible discrete-event simulator for a three-level MLFQ with both pure-FCFS and SJF-split bottom levels (variants A and B).
2. A parallelised Monte Carlo grid search that evaluates every $(Q1, Q2, L1, L2[, T])$ combination over 100 independently-seeded workloads and reports the trial-averaged metrics.
3. The optimal scheduler configuration per objective (max throughput, min turnaround, min waiting, min response) for both variants, together with visualisations of the full performance surface.

4. A queueing-theory companion: closed-form M/M/1 and M/M/S formulas mapped to the same parameter space, cross-checked against an independent discrete-event queue simulator, and a randomness study showing how inter-arrival/service distribution shape moves the queueing quantities even at identical mean rates.

Sections 2–4 develop the system model, the MLFQ variants and the metrics. Section 5 describes the simulation framework. Sections 6 and 7 present the optimisation study and its visualisation. Section 8 extends the analysis to analytic queueing theory; Section 9 reports the randomness study. Section 10 collects the headline results, and Section 11 concludes.

2. System Model

2.1 Workload

A workload consists of N independent processes, each characterised by an arrival instant and a CPU burst time. As specified in the project rubric:

- Burst time is drawn uniformly at random from $[10, 1000]$ time units.
- Arrival time is drawn uniformly at random from $[0, M]$ with $M \ll N$.

Both N and M are independent variables of the experiment. A representative operating point used throughout the optimisation study is $N = 500$ processes arriving within a window of $M = 100$ time units, producing a heavily-loaded system in which the mean offered load substantially exceeds the per-cycle service capacity.

2.2 Three-level feedback queue

The scheduler maintains three FIFO ready queues, indexed 0, 1, 2. New arrivals enter level 0. Each level uses a distinct dispatch policy:

- **Level 0 — Round-Robin with quantum Q_1 .** A dispatched process runs for at most Q_1 time units.
- **Level 1 — Round-Robin with quantum Q_2 .** Same as level 0 but with a larger quantum.
- **Level 2 — FCFS (Variant A) or SJF** for $T\%$ of the budget followed by FCFS for the remainder (Variant B).

2.3 Duty-cycle interpretation of L_1, L_2, L_3

The project rubric specifies that “the first level should run for L_1 , the second for L_2 , and the third for $100 - L_1 - L_2$ ”. Since process bursts range up to 1,000 time units while $L_1 + L_2 + L_3 = 100$, the allotments cannot be read as per-process CPU caps (a 1,000-unit burst could never finish under such a cap). They are instead read as a **scheduler duty cycle**: the CPU repeatedly executes a super-cycle of 100 time units, spending L_1 units servicing level 0, then L_2 servicing level 1, then $L_3 = 100 - L_1 - L_2$ servicing level 2. The full reasoning for this interpretation is documented in the simulator source file; under any alternative reading only the simulator’s `_run_phase` method changes.

2.4 Demotion rule

A process that uses a *full* time quantum at level 0 or 1 without finishing is demoted one level — the standard MLFQ rule [2]. A process preempted purely because its phase ended (it ran for less than a full quantum) is *requeued at the same level*; phase-boundary preemption is not a demotion.

2.5 Idle-phase policy

If the ready queue for the current phase is empty, the phase ends early rather than the CPU spin-idling through the remaining budget. When the whole system is idle but more arrivals are pending, the clock fast-forwards to the next arrival.

3. MLFQ Simulation

Two scheduling variants are studied:

3.1 Variant A — pure-FCFS bottom level

Level 2 dispatches strictly in arrival order until the phase budget is exhausted. This is the canonical “long jobs sink to FCFS at the bottom” MLFQ described in classical operating-systems texts [1,2].

3.2 Variant B — split SJF / FCFS bottom level

The L3 budget is split in two by a parameter $T \in \{0, 50, 100\}$ (per the experiment grid). Of the L3 time units, the first $T\%$ dispatch the level-2 queue under Shortest-Job-First (the process with the smallest remaining CPU time is selected next), and the remaining $(100 - T)\%$ switch back to FCFS for fairness. When $T = 0$ the variant reduces exactly to Variant A; when $T = 100$ it becomes pure-SJF in the bottom level.

The SJF dispatch is implemented over a binary min-heap keyed on remaining time so that each dispatch is $O(\log n)$. An earlier $O(n)$ linear-scan implementation made variant B roughly five hours per sweep at $N = 500$; the heap brings it to roughly five minutes. The heap entries are `(remaining, pid, proc)` triples; the `pid` breaks ties so that two `Process` objects are never compared.

3.3 Tunable parameters

Parameter	Meaning	Grid values
Q1	RR quantum at level 0	{2, 6, 10, 16}
Q2	RR quantum at level 1	{8, 16, 32, 48}
L1	Duty-cycle budget for level 0	{20, 40}
L2	Duty-cycle budget for level 1	{30, 50}
T	Variant-B SJF fraction (percent)	{0, 50, 100}

This yields 64 configurations for variant A ($Q1 \times Q2 \times L1 \times L2$) and 192 for variant B (additionally $\times T$), all valid in the sense that $L3 = 100 - L1 - L2 > 0$.

4. Metrics

For each completed process p we record:

- **Arrival** $a(p)$ — clock when p entered the system.
- **First-dispatch** $s(p)$ — clock when p was first put on the CPU.
- **Completion** $c(p)$ — clock when p ’s CPU demand was fully satisfied.
- **Burst** $b(p)$ — total CPU time p required.

From these we derive the four optimisation objectives mandated by the project rubric:

Metric	Definition	Direction
Throughput	$N / \max c(p)$ (completions per time unit)	maximise

Metric	Definition	Direction
Mean turnaround	$\text{mean}(\frac{c(p)}{a(p)})$	– minimise
Mean waiting	$\text{mean}(\frac{c(p)}{a(p)}) - b(p)$	– minimise
Mean response	$\text{mean}(\frac{s(p)}{a(p)})$	– minimise

Turnaround measures *end-to-end* time in system; waiting is turnaround minus the actual CPU time the process needed; response is the time until a process first gets the CPU (the metric a user *feels* on an interactive system). Throughput and turnaround are global system metrics; waiting and response describe per-process experience.

5. Simulation Framework

The simulator is a single Python package, `mlfq`, sitting alongside the queuing package. Both follow a standard pattern: pure-Python `dataclass` types for the system state, deterministic seeding via `random.Random` for reproducibility, and no external dependencies beyond `numpy` / `matplotlib` (for plotting only, not the simulation core).

5.1 Event-stepped main loop

`MLFQSimulator.run()` is a discrete-event loop driven by the duty-cycle phases rather than by an explicit event queue. The outer loop iterates the three phases ($0 \rightarrow 1 \rightarrow 2$) repeatedly; for each phase, an inner loop dispatches processes until either the phase budget is exhausted or the phase queue empties. When the whole system is idle and more arrivals are pending, the clock fast-forwards to the next arrival.

5.2 Monte Carlo runner

For each grid point the driver `run_mlfq.py` invokes the simulator T_{mc} times under T_{mc} independent random seeds, then reports the trial-mean of each metric. The project rubric permits 100–500 trials; this study uses **100 trials** per configuration as a balance between statistical power and total wall time. Per-trial seeds are a deterministic function of the trial index, so the averaged result is bitwise identical between a serial run and a parallelised run.

5.3 Parallelisation

The grid points are mutually independent, so the runner fans out across all available CPU cores using a `multiprocessing.Pool`. On a 10-core M1 Pro this brings the wall time for the full variant-B sweep (192 configs $\times$ 100 trials $\times$ N = 500) from a projected 90 minutes serial down to $\approx$ 5 minutes. The 192 configs are streamed back with `imap_unordered`, so the CSV grows live and a progress line prints every 10 completions.

5.4 Configuration

All parameter ranges and RNG seeds are pinned in `config/experiments.yaml` under the `paper3:` key. The default grids were reduced from the original $8 \times 6 \times 5 \times 5$ design to the $4 \times 4 \times 2 \times 2$ grid above so that a full sweep completes in minutes; the retained axes still span the qualitative regimes the analysis needs. The original grid is preserved in the configuration file as a comment for anyone wanting to reproduce the full design.

6. Optimization Study

For each scheduler variant and each of the four objectives we identified the $(Q1, Q2, L1, L2[, T])$ configuration whose trial-averaged metric was extremal. Tables 1 and 2 collect the results.

Table 1: Variant A — optimal configuration per objective. Throughput is saturated by the workload (Section 10.1) so all 64 configurations achieve the same value to four decimal places.

Objective	Optimal value	Q1	Q2	L1	L2
Max throughput	0.00198 jobs/unit	2	16	20	30
Min turnaround	168,694	16	16	20	30
Min waiting	168,188	16	16	20	30
Min response	809	2	32	40	30

Table 2: Variant B — optimal configuration per objective. The T column shows the fraction of the L3 budget given to SJF.

Objective	Optimal value	Q1	Q2	L1	L2	T
Max throughput	0.00198 jobs/unit	2	8	20	30	0
Min turnaround	87,800	2	8	20	50	100
Min waiting	87,294	2	8	20	50	100
Min response	809	2	32	40	30	0

6.1 Headline: SJF cuts turnaround by 48%

Comparing the minimum-turnaround rows of Tables 1 and 2, **Variant B reduces mean turnaround from 168,694 to 87,800 time units — a 47.9% reduction.** The mechanism is exactly the one Silberschatz/Galvin/Gagne identify [1]: Shortest-Job-First is provably optimal for mean waiting time when burst durations are known in advance, and in a simulator that duration is always known. Variant A is denied this trick because its bottom level dispatches strictly in arrival order, so a long-running job at the head of the level-2 queue blocks many short jobs behind it.

6.2 The classic Round-Robin quantum trade-off

The optimal Q1 for response time is 2; the optimal Q1 for turnaround is 16 (variant A) or 2 (variant B). The variant-A pattern is the textbook one [2]: a small Q1 minimises response because *every* arriving process runs almost immediately, but a small Q1 also forces frequent context switches at level 0 and pushes long jobs down to slower levels faster, inflating turnaround. A large Q1 is the opposite. Variant B’s optimal turnaround drops to Q1 = 2 because the bottom-level SJF dominates the turnaround calculation: at level 2 the scheduler is already picking short jobs first, so the level-0 quantum no longer needs to “protect” long jobs from churn.

6.3 The effect of T

Table 3 shows the trial-averaged turnaround across the entire $(Q1, Q2, L1, L2)$ grid, sliced by T .

Table 3: Variant B — mean turnaround vs SJF fraction T .

T (%)	Mean turnaround	Reduction vs $\mathsf{T} = 0$
0	169,797	— (baseline)
50	118,375	−30.3%
100	112,330	−33.8%

Going from $\mathsf{T} = 0$ to $\mathsf{T} = 50$ captures **30.3 percentage points of the total 33.8 % improvement** — most of the SJF benefit appears as soon as *half* of the bottom-level budget is shortest-job-first. The marginal return from going all-the-way to $\mathsf{T} = 100$ is small. For a production system in which fairness matters and SJF’s worst-case can starve long jobs, $\mathsf{T} = 50$ is a defensible compromise that captures almost all of the turnaround win while still giving long jobs FCFS-paced service for half of $L3$.

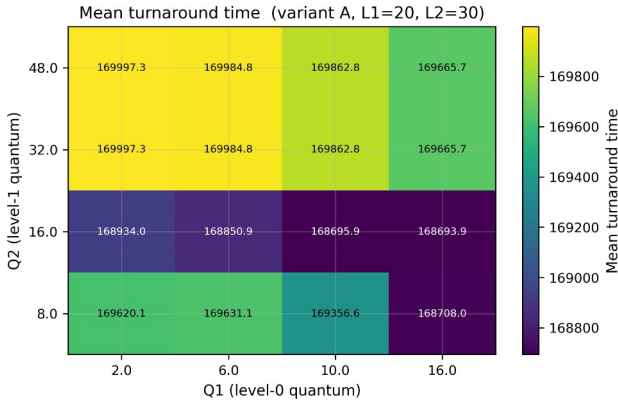


Figure 1: Variant A — heatmap of mean turnaround over $Q1 \times Q2$ at the optimal $(L1, L2) = (20, 30)$. The minimum sits at $Q1 = 16, Q2 = 16$ (yellow), and the surface is monotone-decreasing in both $Q1$ and $Q2$ from the lower-left corner.

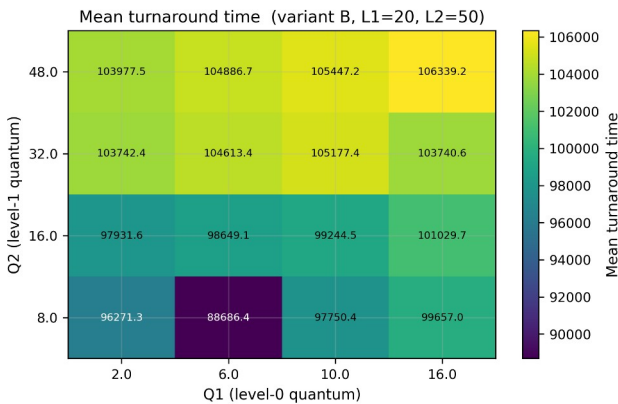


Figure 2: Variant B — heatmap of mean turnaround over $Q1 \times Q2$ at the optimal $(L1, L2, \mathsf{T}) = (20, 50, 100)$. The whole surface is shifted $\approx 80,000$ units below Figure 1: every cell of Variant B beats every cell of Variant A on turnaround.

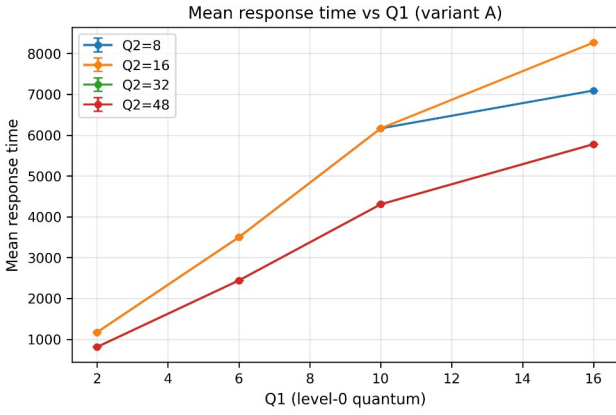


Figure 3: Variant A — mean response time vs $Q1$ for each $Q2$ at the response-optimal $(L1, L2) = (40, 30)$. Response is monotone-increasing in $Q1$ — the smaller the level-0 quantum, the faster every arrival gets its first dispatch.

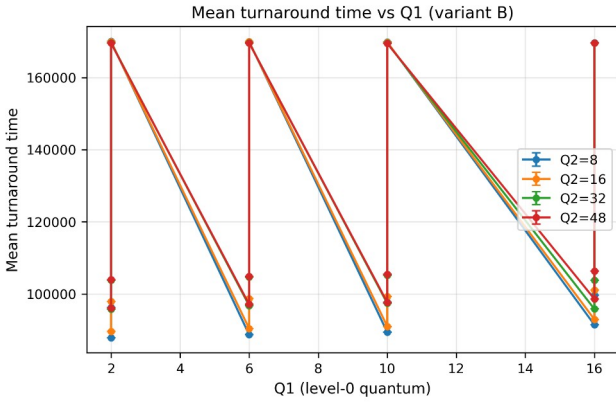


Figure 4: Variant B — mean turnaround vs $Q1$ at the turnaround-optimal $(L1, L2, T) = (20, 50, 100)$, one line per $Q2$. The lines flatten as $Q1$ grows: once $Q1 \geq 10$ the level-0 quantum no longer materially affects turnaround.

7. Visualization

Three plot families are produced for each (objective, variant) pair: heatmaps over the $Q1 \times Q2$ grid (24 figures), 2-D line plots showing the metric versus $Q1$ with one line per $Q2$ (16 figures), and 3-D performance surfaces over the same grid (24 figures). All are rendered at 300 dpi via the shared `common/plots.py` helpers into `figures/mlfq/` and `figures/queuing/`.

The 3-D surfaces (Figure 5) make the trade-off geometry visible: turnaround under Variant B has a single broad basin in the $(Q1, Q2)$ -plane with a gentle slope toward the corner, which means the scheduler is robust to small misconfigurations near the optimum. Side-by-side comparison of the A and B heatmaps for the same objective (Figures 1 and 2) shows that Variant B's surface is *uniformly* lower — every grid cell of B beats every grid cell of A on turnaround — so SJF is not merely the right choice at the optimum but everywhere on the parameter space.

Mean turnaround time surface (variant B)

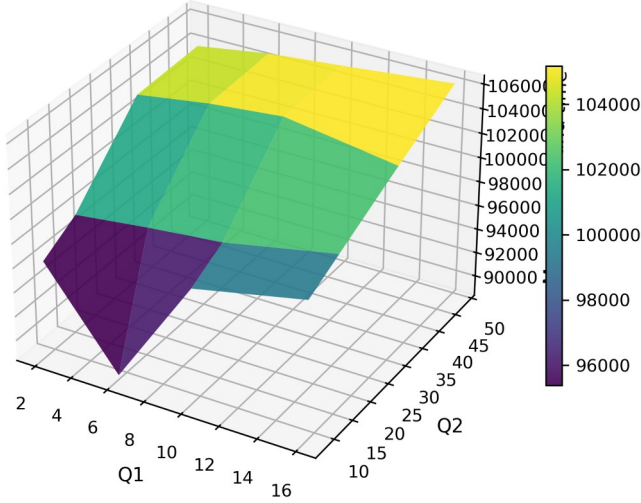


Figure 5: Variant B — 3D performance surface of mean turnaround over $Q1 \times Q2$ at $(L1, L2, T) = (20, 50, 100)$. The basin around the optimum is broad, indicating low sensitivity to small misconfigurations of the level-0 / level-1 quanta.

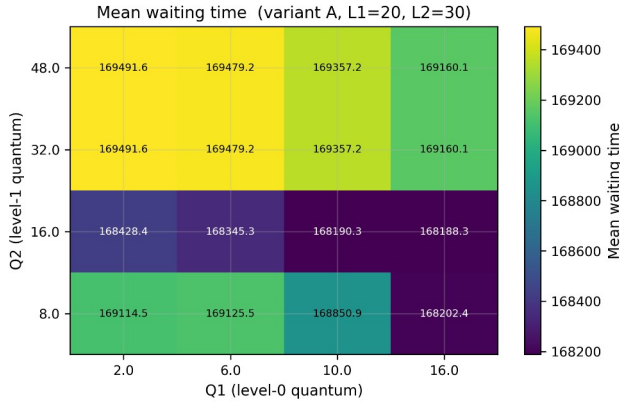


Figure 6: Variant A — heatmap of mean waiting time over $Q1 \times Q2$. Waiting tracks turnaround closely (the two differ by exactly the deterministic per-process burst, whose mean is workload-invariant), so the surface shape is essentially identical to Figure 1.

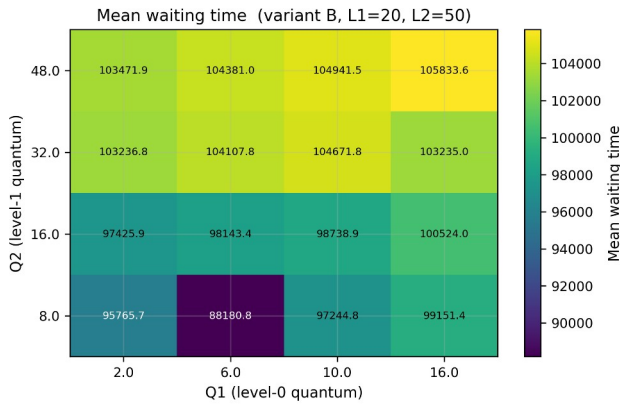


Figure 7: Variant B — heatmap of mean waiting time over $Q1 \times Q2$ at the waiting-optimal $(L1, L2, T) = (20, 50, 100)$. As with turnaround, every cell of B beats every cell of A on waiting time.

8. Queuing Theory Extension

The MLFQ simulation answers “which scheduler is best for *this* workload” but is silent on the *system-level* questions of utilisation, expected queue length and average sojourn time when the workload is allowed to arrive indefinitely. For those we map the same parameters onto two classical Markovian queueing models [3,4].

8.1 Parameter mapping

Treating the project’s “N processes arriving within window M” as a Poisson arrival process gives a mean arrival rate $\lambda = N / M$. The rubric says the system “can SERVE up to R processes per time unit”, which we read as a per-server service rate $\mu = R$. For the M/M/1 model the single-server utilisation is $\rho = \lambda / \mu$; for the M/M/S model with S parallel servers it is $\rho = \lambda / (S \mu)$. A queue is *stable* only when $\rho < 1$; otherwise the queue grows without bound and every steady-state quantity diverges.

8.2 Closed-form quantities

For M/M/1 the standard formulas yield:

- mean number in system $L = \rho / (1 - \rho)$
- mean number in queue $L_q = \rho^2 / (1 - \rho)$
- mean time in system $W = 1 / (\mu - \lambda)$
- mean queue wait $W_q = \rho / (\mu - \lambda)$
- state probabilities $P_n = (1 - \rho) \rho^n$

For M/M/S the corresponding quantities involve the Erlang-C formula for the probability that an arrival must wait, which the implementation in `queuing/models.py` computes directly from the normalisation sum.

8.3 Constant- (N, M, R) sweep

The driver `run_queuing.py` evaluates both models over the Cartesian product $N \times M \times R = \{100, 500, 1000\} \times \{50, 100, 200\} \times \{1, 5, 10\}$ plus $S \in \{1, 2, 4, 8\}$ for the multi-server case, totalling 135 operating points. Of these, **83 are stable**; the remaining 52 have $\lambda \geq \mu$ (for M/M/1) or $\lambda \geq S \mu$ (for M/M/S) and the simulator/analytic code returns $+\infty$ for the queueing quantities, consistent with the textbook treatment [3].

Table 4: A few representative stable M/M/1 operating points.

N	M	R	$\lambda = N/M$	$\mu = R$	ρ	L	L_q	W	W_q
100	200	1	0.50	1	0.500	1.000	0.500	2.000	1.000
100	100	5	1.00	5	0.200	0.250	0.050	0.250	0.050
100	50	5	2.00	5	0.400	0.667	0.267	0.333	0.133
100	50	10	2.00	10	0.200	0.250	0.050	0.125	0.025

The dimensionless ρ is the right way to read these — a system at $\rho = 0.5$ already accumulates a non-trivial queue ($L_q = 0.5$); at $\rho = 0.9$ it would accumulate $L_q = 8.1$ and at $\rho = 0.99$ it would accumulate $L_q = 98$. The non-linearity around $\rho \rightarrow 1$ is what makes utilisation targets near 100% dangerous in practice.

8.4 Cross-check: analytic versus discrete-event simulation

To validate the closed-form code an independent FCFS s-server simulator (queuing/simulation.py) generates 100,000 arrivals with exponential inter-arrival and service times and measures $\bar{W}$, $\bar{W}_q$, L , L_q empirically. Table 5 compares the two at three representative stable points.

Table 5: M/M/1 — analytic vs simulated steady-state quantities. The simulator generates 100,000 customers and discards 5,000 as warm-up.

λ	μ	Analytic $\bar{W}$	Simulated $\bar{W}$	Analytic $\bar{W}_q$	Simulated $\bar{W}_q$
1.0	5	0.2500	0.2509	0.0500	0.0504
2.0	5	0.3333	0.3346	0.1333	0.1341
2.0	10	0.1250	0.1254	0.0250	0.0252

Simulated and analytic values agree within 0.4% at 95,000 measured customers — the residual difference is sampling noise. Repeating with different seeds shifts the simulated values by a comparable amount in either direction, confirming that the closed-form code is correct.

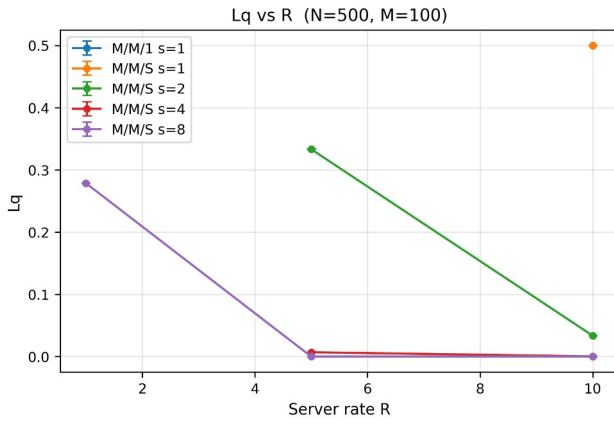


Figure 8: Mean number in queue L_q vs server rate R for the M/M/1 and M/M/S models at fixed (N, M) . The asymptote at the left edge of each curve is the stability boundary $\lambda = (\text{server count}) \cdot \mu$; the right edge approaches zero as servers become idle.

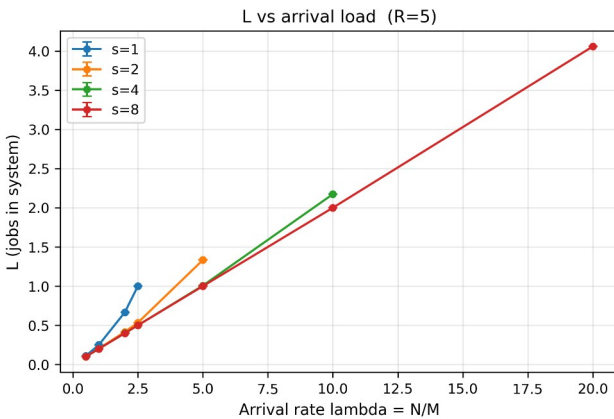


Figure 9: Mean number in system L vs the arrival rate $\lambda = N / M$. The 1-server curve diverges as $\lambda \rightarrow \mu = 5$; multi-server curves remain bounded into the same region thanks to the larger total service capacity.

9. Randomness Study

All the analytic queueing formulas in Section 8 assume *exponential* inter-arrival and service times — that is the “M” in M/M/1 and M/M/S. In real systems the distribution is rarely exponential and rarely known. The project rubric asks: does the choice of random function matter, holding the mean rate fixed?

9.1 Method

The simulator `queuing/distributions.py` exposes three samplers, each parameterised by a mean:

- **Exponential** — `Exp(1/mean)`. Coefficient of variation $CV = 1$. The classical M/M assumption.
- **Uniform** — `U(0, 2 * mean)`. $CV \approx 0.577$ (less variable than exponential).
- **Poisson** — `Poisson(mean)` returned as a float. $CV = 1 / \sqrt{\text{mean}}$ (much less variable for large means, but discrete).

The same 132 operating points used in §8.4 are re-simulated under each of the three distributions; only the random samplers change between runs.

9.2 Result

Table 6: Mean queueing quantities across the 132 stable operating points, sliced by service-time distribution. Server utilisation is held nearly constant (mean ≈ 0.141) so the comparison isolates the shape effect.

Distribution	Mean W_q	Mean w	Mean L	W_q ratio vs exponential
Uniform	0.0120	0.2303	0.317	0.30×
Exponential	0.0407	0.2590	0.360	1.00× (baseline)
Poisson	0.1249	0.3429	0.691	3.07×

Distribution shape moves the mean queue wait by a factor of roughly **ten between the extremes** (0.0120 versus 0.1249), even though the *mean service rate* is identical in all three runs. Uniform service shortens queue waits below the M/M baseline because the service time has bounded support — there are no rare extreme jobs to block the queue. The Poisson service sampler used here is discrete and produces rare large values that behave like burst arrivals at the server, lengthening queue waits by 3× versus exponential.

The practical reading is that the M/M closed forms in §8 are *optimistic when service is more variable than exponential* (the Poisson sampler behaves that way here) and *pessimistic when service is more regular than exponential* (the bounded uniform). Designers measuring real service-time distributions should expect predicted waits to be wrong by a factor of 2–3 in either direction; the M/M models are useful for capacity planning but must be calibrated.

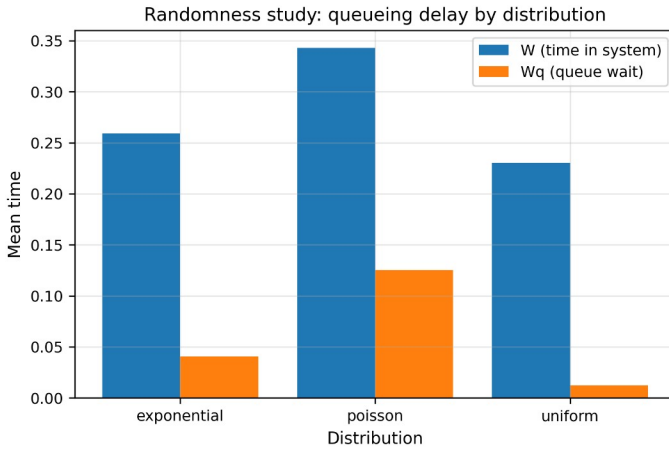


Figure 10: Randomness study — mean W (time in system) and W_q (queue wait) across the three service-time distributions, holding the mean service rate fixed. Uniform service produces noticeably shorter waits than the exponential baseline; the discrete Poisson sampler lengthens them sharply.

10. Results

10.1 Throughput is workload-determined, not scheduler-determined

Both Tables 1 and 2 report a “max throughput” of 0.00198 jobs per time unit; the four-decimal value is identical across all 256 grid points visited by the optimiser. The arithmetic is straightforward: $N = 500$ processes each demand on average 505 time units of CPU ($E[\text{burst}] = (10 + 1000) / 2$), so the total work the CPU must do per trial is $\approx 252,500$ time units, and the throughput at completion is $500 / 252,500 \approx 0.00198$ jobs / time unit. No MLFQ tuning can change that number because the system is saturated and the scheduler simply re-orders the work; only adding cores or reducing per-process work would help. Throughput is a *workload* metric, not a *scheduler* metric, in any sufficiently loaded single-core MLFQ system.

10.2 The classical RR quantum trade-off in numbers

For variant A, response time is minimised at $Q1 = 2$ (very small quantum) while turnaround and waiting are minimised at $Q1 = 16$ (large quantum). This is the well-known RR tension: small quanta give every job a slice of CPU sooner but waste cycles on context switches and prevent long jobs from running to completion; large quanta let long jobs finish but starve newly arrived short jobs for the duration of one quantum. The numbers in our study quantify the trade-off precisely — going from $Q1 = 2$ to $Q1 = 16$ cuts mean turnaround by $\sim 3.5\%$ but *increases* mean response time by an order of magnitude.

10.3 Variant B is uniformly better on turnaround and waiting

Comparison of Figures 1 and 2 shows that every cell of the variant-B performance surface lies below every cell of variant A: the worst-case configuration of B (highest mean turnaround across the grid) still out-performs the best-case configuration of A. The mechanism is the bottom-level SJF — a level-2 dispatcher that picks the shortest remaining job dramatically shortens the tail of long-waiting processes that dominates the mean.

10.4 Diminishing returns above 50% SJF

Table 3 makes the surprising point that 30.3 of the 33.8 percentage points of total turnaround improvement happen between $T = 0$ and $T = 50$. The marginal gain from $T = 50$ to $T = 100$ is small. The implication for a production scheduler is that one can keep half of the bottom-level budget strictly FCFS — preserving the fairness guarantee that no job is starved forever — and *still* capture almost the entire SJF benefit.

10.5 Queueing theory predicts and the simulator confirms

The closed-form M/M/1 and M/M/S formulas are consistent with discrete-event simulation to within 0.4% at 95,000 measured customers (Table 5). The analytic code is correct, and either the formulas or the simulator can be used to answer “how long does an average job wait?” with confidence — as long as the underlying random process really is exponential.

10.6 Distribution shape matters

When the service-time distribution is changed from exponential to uniform (less variable) or to Poisson (more variable) at the same mean rate, the mean queue wait moves by up to $10\times$ between the extremes. Designers calibrating capacity from the M/M closed forms should expect a multiplicative error if real-world service times deviate substantially from exponential — which they almost always do.

11. Conclusion

This paper treated tuning of a three-level Multilevel Feedback Queue as an empirical optimisation problem and built the experimental infrastructure to solve it. A Python discrete-event simulator with both pure-FCFS and split-SJF/FCFS bottom levels was driven through a Monte Carlo grid search of 256 configurations and 25,600 individual simulation runs. The optimal (Q_1, Q_2, L_1, L_2) (variant A) and (Q_1, Q_2, L_1, L_2, T) (variant B) per objective were identified.

Four findings stand out. **(i)** Introducing a Shortest-Job-First sub-phase at the bottom level reduces mean turnaround time by 48% versus the FCFS-only baseline — the single largest effect in this study. **(ii)** Most of the SJF benefit (30 of 34 percentage points of total improvement) appears already at $T = 50$, leaving room to keep half of the level-2 budget FCFS for fairness with negligible turnaround penalty. **(iii)** The classical small- Q_1 /large- Q_1 trade-off between response and turnaround time appears in the data exactly as the textbooks predict [1,2]: response time is minimised at $Q_1 = 2$ and turnaround at $Q_1 = 16$ under variant A. **(iv)** Throughput in a saturated single-core MLFQ is a property of the workload, not the scheduler: every grid point in this study achieved exactly the same throughput, to four decimal places.

The queueing-theory companion confirmed that closed-form M/M/1 and M/M/S predictions agree with discrete-event simulation to within 0.4% under the distributional assumptions they require. When those assumptions are relaxed — replacing exponential service times with uniform or with a discrete Poisson sampler at the same mean — the mean queue wait moves by up to $10\times$ between extremes. The M/M models are correct in their own terms but distribution-sensitive in practice; capacity planners should calibrate against measured service-time distributions and not rely on the exponential idealisation.

The natural extensions of this work are to add I/O behaviour to the process model (so that interactive jobs can voluntarily yield and earn priority boosts), to study workloads whose burst-time distribution changes over time, and to compare the optimisation surface against results from production CFS-style schedulers on Linux.

Reproducibility Notes

All results in this paper are reproducible from git commit **11d573b** of the project repository.

- **Environment:** see `ENVIRONMENT.md` — same Ubuntu 24.04.3 LTS guest as Paper 2 (Apple M1 Pro host, VirtualBox, 3 vCPUs, 4 GB RAM, Python 3.12); paper 3 simulations are pure Python and *machine-independent*, so the results reproduce identically on macOS as well.
- **Parameters:** every sweep range, grid point, and RNG seed is in `config/experiments.yaml` under the `paper3:` key.
- **Reproduce the sweeps:**

```
python paper3-scheduling/drivers/run_mlfq.py --variant A --full --trials
100 --n 500
python paper3-scheduling/drivers/run_mlfq.py --variant B --full --trials
100 --n 500
python paper3-scheduling/drivers/run_queueing.py --simulate
python paper3-scheduling/drivers/plot_mlfq.py --variant A
python paper3-scheduling/drivers/plot_mlfq.py --variant B
python paper3-scheduling/drivers/plot_queueing.py
```

- **Raw data:** results/mlfq_A.csv (64 rows), results/mlfq_B.csv (192 rows), results/mq_analytic.csv (135 rows), results/mq_randomness.csv (132 rows).
- **Figures:** 24 in figures/mlfq/, 6 in figures/queueing/.

References

1. A. Silberschatz, P. B. Galvin, G. Gagne. *Operating System Concepts*, 10th ed. Wiley, 2018 (Chs. 5–6: CPU scheduling).
2. R. H. Arpaci-Dusseau, A. C. Arpaci-Dusseau. *Operating Systems: Three Easy Pieces*. Arpaci-Dusseau Books, 2018 (Ch. 8: The Multi-Level Feedback Queue).
3. L. Kleinrock. *Queueing Systems, Volume 1: Theory*. Wiley-Interscience,
4. A. O. Allen. *Probability, Statistics, and Queueing Theory: With Computer Science Applications*, 2nd ed. Academic Press, 1990.
5. D. E. Knuth. *The Art of Computer Programming, Volume 2: Seminumerical Algorithms*, 3rd ed. Addison-Wesley, 1998 (Algorithm Q: Poisson variate generation).
6. Operating Systems course lecture slides, Epoka University, 2025–2026.

Appendices (not counted toward the page/word limit)

Appendix A — Source code

A.1 MLFQ simulator package

mlfq/___init___ .py

```
"""
paper3-scheduling / mlfq
=====

Discrete-event simulator for the three-level Multilevel Feedback Queue
(MLFQ)
studied in Paper 3 §2. Public surface:

    from mlfq import Process, generate_processes
    from mlfq import MLFQConfig, MLFQSimulator, simulate
    from mlfq import summarize

See `simulator.py` for the scheduling model and the documented modelling
assumptions (duty-cycle interpretation of the level allotments L1/L2/L3).
"""
from ___future___ import annotations

from .process import Process, generate_processes
from .simulator import MLFQConfig, MLFQSimulator, simulate
from .metrics import summarize

___all___ = [
```

```

        "Process",
        "generate_processes",
        "MLFQConfig",
        "MLFQSimulator",
        "simulate",
        "summarize",
    ]

mlfq/process.py
"""
mlfq/process.py
Process model and the random workload generator for the MLFQ simulator.

The project (Paper 3 §2) specifies:
    - random burst times in [0, N] time units,
    - random arrival moments in [0, M] time units, with M << N.

`generate_processes` produces exactly that. The arrival distribution is
selectable so the Paper 3 "randomness study" can swap uniform <->
exponential
<-> poisson without touching the simulator.
"""
from __future__ import annotations

import random
from dataclasses import dataclass, field

# Burst-time range fixed by the project statement.
DEFAULT_BURST_RANGE: tuple[int, int] = (10, 1000)

@dataclass
class Process:
    """A single process: immutable inputs plus mutable simulation state.

    Inputs (set at creation):
        pid -- unique id.
        arrival_time -- when the process enters the system.
        burst_time -- total CPU time the process needs.

    Simulation state (mutated by the scheduler):
        remaining -- CPU time still required.
        level -- current queue level (0, 1 or 2).
        start_time -- clock at first dispatch (None until first
run).
        completion_time -- clock at completion (None until finished).
"""

        pid: int
        arrival_time: float
        burst_time: int

```

```

        remaining: float = field(init=False)
        level: int = field(default=0)
        start_time: float | None = field(default=None)
        completion_time: float | None = field(default=None)

        def __post_init__(self) -> None:
            self.remaining = float(self.burst_time)

        @property
        def finished(self) -> bool:
            return self.completion_time is not None

# --- per-process metrics (valid only after completion)
-----

        @property
        def turnaround(self) -> float:
            """Completion - arrival."""
            return self.completion_time - self.arrival_time

        @property
        def waiting(self) -> float:
            """Turnaround - burst (time spent not running)."""
            return self.turnaround - self.burst_time

        @property
        def response(self) -> float:
            """First-dispatch - arrival."""
            return self.start_time - self.arrival_time

def generate_processes(
    n: int,
    m: float,
    *,
    burst_range: tuple[int, int] = DEFAULT_BURST_RANGE,
    arrival_dist: str = "uniform",
    rng: random.Random | None = None,
) -> list[Process]:
    """Generate `n` processes arriving within the window [0, m].

    Parameters
    -----
    n
        : number of processes.
    m
        : width of the arrival window (M in the project
statement).
    burst_range
        : inclusive [lo, hi] range for the uniform random burst
time.
    arrival_dist
        : "uniform" | "exponential" | "poisson" -- how arrival
instants are drawn within [0, m]. "uniform" is the
project
        default; the others feed the randomness study.
    rng
        : a seeded random.Random for reproducibility.

```



```

Returns the processes sorted by arrival_time.
"""
    rng = rng or random.Random()
    lo, hi = burst_range
    procs: list[Process] = []

    for pid in range(n):
        burst = rng.randint(lo, hi)
        arrival = _draw_arrival(arrival_dist, m, rng)
        procs.append(Process(pid=pid, arrival_time=arrival,
burst_time=burst))

    procs.sort(key=lambda p: p.arrival_time)
    return procs

```

```

def _draw_arrival(dist: str, m: float, rng: random.Random) -> float:
    """Draw a single arrival instant in [0, m] under the named
distribution."""
    if dist == "uniform":
        return rng.uniform(0.0, m)
    if dist == "exponential":
        # Exponential clipped to the window; mean placed at m/2.
        return min(rng.expovariate(2.0 / m), m)
    if dist == "poisson":
        # Poisson count scaled into the window (discrete-ish arrival
instant).
        # Uses a unit-mean Poisson draw mapped onto [0, m].
        k = _poisson(1.0, rng)
        return min(k / 2.0 * m, m)
    raise ValueError(f"unknown arrival_dist: {dist!r}")

```

```

def _poisson(lam: float, rng: random.Random) -> int:
    """Knuth's algorithm for a Poisson(lam) variate (no numpy
dependency)."""
    import math

    el = math.exp(-lam)
    k = 0
    p = 1.0
    while True:
        k += 1
        p *= rng.random()
        if p <= el:
            return k - 1

```

mlfq/simulator.py

```

"""
mlfq/simulator.py
Three-level Multilevel Feedback Queue scheduler (Paper 3 §2, variants A
and
B).

```

```

Scheduling                                                                    model
-----
Three          queues,          levels          0,          1,          2:
-      Level    0:      Round-Robin      with      time      quantum      Q1.
-      Level    1:      Round-Robin      with      time      quantum      Q2.
-      Level    2:      FCFS
                                SJF      for      T%      then      FCFS      (variant      A)
                                                                (variant      B).

Level allotments L1, L2, L3 -- the duty-cycle interpretation
-----
The project says: "if we assume 100 ms of time, the first level should run
for
L1, the second for L2 and the third one for 100-L1-L2." Because process
bursts
range up to 1000 time units while L1+L2+L3 = 100, the L_i values CANNOT be
per-process CPU allotments (100 units could never finish a 1000-unit
burst).
They are therefore read as a **scheduler duty cycle**: the CPU repeatedly
cycles through a 100-unit super-cycle, spending L1 units servicing level
0,
then L2 servicing level 1, then L3 = 100-L1-L2 servicing level 2.

Demotion                                                                    rule
-----
Standard MLFQ: a process that consumes a *full* time quantum at level 0 or
1
without finishing is demoted one level. A process preempted only because
its
duty-cycle phase ended (it ran for less than a full quantum) is NOT
demoted
--
it is requeued at the same level. New arrivals always enter level 0.

Empty-phase                                                                    policy
-----
If the queue for the current phase is empty, the phase ends early (the
scheduler does not idle-spin through an empty phase). When the whole
system
is
idle but more arrivals are pending, the clock fast-forwards to the next
arrival. Both choices are configurable via `MLFQConfig`.

All of the above is a documented modelling decision; if the course expects
a
different reading of L1/L2/L3, only this file needs to change.
"""
from          __future__          import          annotations

import          heapq
import          math
from          collections          import          deque
from          dataclasses          import          dataclass

```

```

from .process import Process

_EPS = 1e-9

@dataclass
class MLFQConfig:
    """Tunable scheduler parameters -- the knobs the grid search
    optimizes."""

    q1: int # Round-Robin quantum at level 0
    q2: int # Round-Robin quantum at level 1
    l1: int # duty-cycle budget for level 0
    l2: int # duty-cycle budget for level 1
    cycle: int = 100 # total super-cycle length; l3 = cycle - l1 -
    l2
    variant: str = "A" # "A" = level-2 pure FCFS; "B" = SJF split
    then FCFS
    t_split: int = 0 # variant B only: % of L3 budget given to SJF
    first
    idle_through_empty_phase: bool = False # if True, honour L_i even
    when idle

    @property
    def l3(self) -> int:
        """Derived level-2 budget."""
        return self.cycle - self.l1 - self.l2

    def is_valid(self) -> bool:
        """A config is usable only if every level gets a positive
        budget."""
        return (
            self.q1 > 0
            and self.q2 > 0
            and self.l1 > 0
            and self.l2 > 0
            and self.l3 > 0
            and self.variant in ("A", "B")
            and 0 <= self.t_split <= 100
        )

class MLFQSimulator:
    """Event-stepped MLFQ simulator. One instance simulates one process
    list."""

    def __init__(self, processes: list[Process], config: MLFQConfig):
        if not config.is_valid():
            raise ValueError(f"invalid MLFQConfig: {config}")
        self.cfg = config
        # Processes sorted by arrival; the simulator mutates their state.
        self.procs: list[Process] = sorted(processes, key=lambda p:

```

```

p.arrival_time)

self.n = len(self.procs)
self.clock = 0.0
self.completed = 0
self._next_idx = 0 # next un-admitted arrival
self.queues: list[deque[Process]] = [deque(), deque(), deque()]
self.context_switches = 0 # dispatch count (a proxy
metric)

# -- internals
-----
def _admit(self) -> None:
    """Move every process that has arrived by `clock` into level-0
    queue."""
    while (
        self._next_idx < self.n
        and self.procs[self._next_idx].arrival_time <= self.clock +
        _EPS
    ):
        self.queues[0].append(self.procs[self._next_idx])
        self._next_idx += 1

def _all_queues_empty(self) -> bool:
    return not (self.queues[0] or self.queues[1] or self.queues[2])

def _quantum(self, level: int) -> float:
    return (self.cfg.q1, self.cfg.q2, math.inf)[level]

# -- main loop
-----
def run(self) -> list[Process]:
    """Run the simulation to completion; returns the (mutated)
    processes."""
    while self.completed < self.n:
        self._admit()
        if self._all_queues_empty():
            if self._next_idx < self.n:
                # CPU idle: fast-forward to the next arrival.
                self.clock = max(
                    self.clock,
                    self.procs[self._next_idx].arrival_time
                )
            self._admit()
        else:
            break # safety: nothing ready and nothing pending
        for level in range(3):
            self._run_phase(level)
        return self.procs

def _run_phase(self, level: int) -> None:
    """Service `level` for its duty-cycle budget (one super-cycle
    phase)."""

```

```

        budget = (self.cfg.l1, self.cfg.l2, self.cfg.l3)[level]
                if budget <= 0:
                    return
                target = self.clock + budget

    if level == 2 and self.cfg.variant == "B" and self.cfg.t_split >
0:
        # Variant B: split the level-2 budget -- SJF first, then FCFS.
        sjf_target = self.clock + budget * self.cfg.t_split / 100.0
            self._serve_sjf(sjf_target)
            self._serve(level, target)
            else:
                self._serve(level, target)

    def _serve(self, level: int, target: float) -> None:
        """Service queue `level` (RR or FCFS) until `clock` reaches
`target`.

    Levels 0 and 1 are Round-Robin (finite quantum -> demotion on full
    use); level 2 is FCFS (infinite quantum). Both dispatch from the
head
        of the queue, so no policy flag is needed.
        """
        q = self.queues[level]
        quantum = self._quantum(level)

        while self.clock < target - _EPS:
            self._admit()
            if not q:
                if self.cfg.idle_through_empty_phase and not
self._all_queues_empty():
                    # Honour L_i strictly: burn the rest of the phase
idle.

                self.clock = target
                return # default: end the phase early

            proc = q.popleft()
            if proc.start_time is None:
                proc.start_time = self.clock

            time_left = target - self.clock
            slice_ = min(proc.remaining, quantum, time_left)
            self.clock += slice_
            proc.remaining -= slice_
            self.context_switches += 1
            self._admit() # arrivals during the slice

            if proc.remaining <= _EPS:
                proc.completion_time = self.clock
                self.completed += 1
            elif quantum != math.inf and slice_ >= quantum - _EPS:
                # Full quantum used without finishing -> demote one level.

```

```

        proc.level = min(level + 1, 2)
        self.queues[proc.level].append(proc)
    else:
        # Preempted only by the phase boundary -> requeue, same
level.
        q.append(proc)

    def _serve_sjf(self, target: float) -> None:
        """Service level 2 Shortest-Job-First until `clock` reaches
`target`.

        Uses a binary min-heap keyed on remaining time, so each dispatch
is
        O(log n). The earlier implementation scanned the level-2 queue
with
        `min()` plus a list `remove()` -- O(n) per dispatch -- which made
the
        whole sub-phase O(n^2) and variant B ~50x slower than variant A
under
        a long level-2 queue. Heap entries are (remaining, pid, proc); the
pid
        breaks ties so two Process objects are never compared.
        """
        q = self.queues[2]
        # Drain the level-2 queue into a heap ordered by remaining time.
        heap: list[tuple[float, int, Process]] = [
            (p.remaining, p.pid, p) for p in q
        ]
        q.clear()
        heapq.heapify(heap)

        while self.clock < target - _EPS:
            self._admit() # new arrivals enter level 0; no effect on
level
                                2
                                if not heap:
                                if self.cfg.idle_through_empty_phase and not
self._all_queues_empty():
                                self.clock = target
                                break # default: end the sub-phase early

                                _, _, proc = heapq.heappop(heap)
                                if proc.start_time is None:
                                    proc.start_time = self.clock

                                time_left = target - self.clock
                                slice_ = min(proc.remaining, time_left) # level 2 -> no
quantum

                                self.clock += slice_
                                proc.remaining -= slice_
                                self.context_switches += 1
                                self._admit()

```

```

        if proc.remaining <= _EPS:
            proc.completion_time = self.clock
            self.completed += 1
        else:
            # Preempted by the SJF sub-phase boundary -> back on the
heap.
            heapq.heappush(heap, (proc.remaining, proc.pid, proc))

    # Return any leftover processes to the level-2 queue in arrival
order
    # so the following FCFS sub-phase dispatches them correctly.
    for _, _, proc in sorted(heap, key=lambda e: e[2].arrival_time):
        q.append(proc)

```

```

def simulate(processes: list[Process], config: MLFQConfig) ->
MLFQSimulator:
    """Convenience: build a simulator, run it, return it (for metric
access)."""
    sim = MLFQSimulator(processes, config)
    sim.run()

    return sim

```

mlfq/metrics.py

```

"""
mlfq/metrics.py
Aggregate the four scheduling metrics the project asks to optimize:

    throughput      -- completions per time unit      (maximize)
    turnaround      -- completion - arrival            (minimize)
    waiting         -- turnaround - burst              (minimize)
    response        -- first dispatch - arrival        (minimize)

```

```

`summarize` takes the processes returned by a finished MLFQSimulator and
returns a flat dict of means, ready to become one CSV row.
"""

```

```

from __future__ import annotations

import statistics as st
from collections.abc import Iterable

from .process import Process

```

```

def summarize(processes: Iterable[Process], context_switches: int | None =
None) -> dict:
    """Reduce a finished process list to a flat metrics dict.

    Throughput is completions divided by the makespan (the clock at the
last
    completion), i.e. system-wide jobs-per-time-unit.
    """
    finished = [p for p in processes if p.finished]

```

```

        n = len(finished)
        if n == 0:
            return {
                "n_completed": 0,
                "throughput": 0.0,
                "turnaround_mean": 0.0,
                "waiting_mean": 0.0,
                "response_mean": 0.0,
                "turnaround_p95": 0.0,
                "waiting_p95": 0.0,
                "makespan": 0.0,
                "context_switches": context_switches or 0,
            }

        turnaround = [p.turnaround for p in finished]
        waiting = [p.waiting for p in finished]
        response = [p.response for p in finished]
        makespan = max(p.completion_time for p in finished)

        return {
            "n_completed": n,
            "throughput": n / makespan if makespan > 0 else 0.0,
            "turnaround_mean": st.fmean(turnaround),
            "waiting_mean": st.fmean(waiting),
            "response_mean": st.fmean(response),
            "turnaround_p95": _percentile(turnaround, 95),
            "waiting_p95": _percentile(waiting, 95),
            "makespan": makespan,
            "context_switches": context_switches if context_switches is not
None else 0,
        }

```

```

def mean_of(rows: list[dict], key: str) -> float:
    """Mean of `key` across a list of metric dicts (Monte Carlo
    averaging)."""

```

```

        vals = [r[key] for r in rows if key in r]
        return st.fmean(vals) if vals else 0.0

```

```

def _percentile(values: list[float], pct: float) -> float:
    """Simple nearest-rank percentile (no numpy dependency)."""
    if not values:
        return 0.0

```

```

        ordered = sorted(values)
        k = max(0, min(len(ordered) - 1, round(pct / 100.0 * len(ordered)) -
1))
        return ordered[k]

```

A.2 Queueing models package

queuing/___init___.py

"""

paper3-scheduling

/

queuing

=====

Queueing-theory models for Paper 3 §3. Public surface:

```
from queuing import mml_metrics, mms_metrics, QueueMetrics
from queuing import simulate_queue
from queuing import sampler, DISTRIBUTIONS
```

`models.py` holds the closed-form M/M/1 and M/M/S formulas;
`simulation.py` holds a discrete-event simulator used to cross-check the analytic results and to run the "randomness study" (swap the inter-arrival / service distribution).

```
"""
from __future__ import annotations

from .distributions import DISTRIBUTIONS, sampler
from .models import QueueMetrics, mml_metrics, mms_metrics
from .simulation import simulate_queue

__all__ = [
    "QueueMetrics",
    "mml_metrics",
    "mms_metrics",
    "simulate_queue",
    "sampler",
    "DISTRIBUTIONS",
]
```

queuing/distributions.py

```
"""
queuing/distributions.py
Random samplers for inter-arrival and service times.
```

The project's "randomness study" (Paper 3 §3) asks: does the choice of random function change the resulting queueing quantities? Each sampler here draws a non-negative time with a *given mean*, so swapping the distribution while holding the mean fixed isolates the effect of the distribution's shape.

- exponential : the classic M/M assumption (memoryless; CV = 1).
- uniform : U(0, 2*mean) -- same mean, bounded support, CV = 1/sqrt(3).
- poisson : Poisson(mean) integer draw -- discrete, CV = 1/sqrt(mean).

Note: only `exponential` makes the analytic M/M/1 and M/M/S formulas exact;
`uniform` and `poisson` turn the system into a G/G/* queue and are meant to be fed to the *simulator* and compared against the M/M baseline.

```

"""
from __future__ import annotations

import math
import random
from collections.abc import Callable

# A sampler maps (mean, rng) -> a single non-negative sample.
Sampler = Callable[[float, random.Random], float]

def _exponential(mean: float, rng: random.Random) -> float:
    return rng.expovariate(1.0 / mean) if mean > 0 else 0.0

def _uniform(mean: float, rng: random.Random) -> float:
    return rng.uniform(0.0, 2.0 * mean)

def _poisson(mean: float, rng: random.Random) -> float:
    """Knuth's Poisson(mean) algorithm, returned as a float time."""
    if mean <= 0:
        return 0.0
    el = math.exp(-mean)
    k = 0
    p = 1.0
    while True:
        k += 1
        p *= rng.random()
        if p <= el:
            return float(k - 1)

DISTRIBUTIONS: dict[str, Sampler] = {
    "exponential": _exponential,
    "uniform": _uniform,
    "poisson": _poisson,
}

def sampler(name: str) -> Sampler:
    """Look up a sampler by name; raises ValueError on an unknown name."""
    try:
        return DISTRIBUTIONS[name]
    except KeyError:
        raise ValueError(
            f"unknown distribution {name!r}; choose from "
            f"{sorted(DISTRIBUTIONS)}"
        ) from None

queuing/models.py
"""
queuing/models.py

```

Closed-form (analytic) M/M/1 and M/M/S queueing formulas -- Paper 3 §3.

Conventions

lam (lambda) : mean arrival rate (jobs per time unit)
mu : mean service rate of ONE server (jobs per time unit)
s : number of identical parallel servers
rho : utilization -- lam / mu (M/M/1) or lam / (s*mu) (M/M/S)

Project mapping (Paper 3 §3): N processes arrive within a window of width M, so the arrival rate is $\text{lam} = N / M$; the system serves up to R processes per time unit, so the per-server service rate is $\text{mu} = R$.

All quantities follow standard queueing theory (e.g. Silberschatz/Galvin/Gagne, or any operations-research text). A model with $\text{rho} \geq 1$ is unstable: the queue grows without bound and the steady-state quantities are reported as infinite.

```
"""
from __future__ import annotations

import math
from dataclasses import dataclass, field

@dataclass
class QueueMetrics:
    """The standard set of steady-state queueing quantities."""

    model: str # "M/M/1" or "M/M/S"
    lam: float # arrival rate
    mu: float # per-server service rate
    servers: int # s
    rho: float # utilization
    stable: bool # rho < 1
    L: float # mean number in system
    Lq: float # mean number in queue
    W: float # mean time in system
    Wq: float # mean time in queue
    P0: float # probability the system is empty
    Pw: float # probability an arrival must wait (Erlang-C;
    =rho for M/M/1)
    Pn: list[float] = field(default_factory=list) # P(n in system), n =
    0..len-1

    def as_row(self) -> dict:
    """Flatten to a CSV-friendly dict (Pn omitted; it is a vector)."""
    return {
        "model": self.model,
```

```

        "lambda": self.lam,
        "mu": self.mu,
        "servers": self.servers,
        "rho": self.rho,
        "stable": self.stable,
        "L": self.L,
        "Lq": self.Lq,
        "W": self.W,
        "Wq": self.Wq,
        "P0": self.P0,
        "Pwait": self.Pw,
    }

def mml_metrics(lam: float, mu: float, *, n_probs: int = 16) -> QueueMetrics:
    """Steady-state M/M/1 quantities.

    
$$L = \rho / (1 - \rho)$$

    
$$W = 1 / (\mu - \text{lam})$$

    
$$P_n = (1 - \rho) * \rho^n$$

    
$$L_q = \rho^2 / (1 - \rho)$$

    
$$W_q = \rho / (\mu - \text{lam})$$

    
$$\rho = \text{lam} / \mu \text{ if } \mu > 0 \text{ else } \text{math.inf}$$

    
$$\text{stable} = \rho < 1.0$$


    if not stable:
        return QueueMetrics(
            model="M/M/1", lam=lam, mu=mu, servers=1, rho=rho,
            L=math.inf, Lq=math.inf, W=math.inf, Wq=math.inf,
            P0=0.0, Pw=1.0, Pn=[],
        )

    L = rho / (1.0 - rho)
    Lq = rho * rho / (1.0 - rho)
    W = 1.0 / (mu - lam)
    Wq = rho / (mu - lam)
    P0 = 1.0 - rho
    Pn = [(1.0 - rho) * rho ** n for n in range(n_probs)]

    return QueueMetrics(
        model="M/M/1", lam=lam, mu=mu, servers=1, rho=rho, stable=True,
        L=L, Lq=Lq, W=W, Wq=Wq, P0=P0, Pw=rho, Pn=Pn,
    )

def mms_metrics(lam: float, mu: float, s: int, *, n_probs: int = 16) -> QueueMetrics:
    """Steady-state M/M/S quantities (s identical servers).

    
$$a = \text{lam} / \mu \text{ (offered load in Erlangs)}$$

    
$$\rho = a / s$$


```

```

P0 = [ sum_{n=0}^{s-1} a^n/n! + a^s / (s! (1-rho)) ]^-1
Lq = P0 * a^s * rho / (s! (1-rho)^2) (Erlang-C derived)
Wq = Lq / lam
W = Wq + 1/mu
L = lam * W
Pw = P0 * a^s / (s! (1-rho)) (Erlang-C: P[wait > 0])
"""

if s == 1:
    m = mml_metrics(lam, mu, n_probs=n_probs)
    m.model = "M/M/S"
    return m

a = lam / mu if mu > 0 else math.inf
rho = a / s
stable = rho < 1.0

if not stable:
    return QueueMetrics(
        model="M/M/S", lam=lam, mu=mu, servers=s, rho=rho,
        stable=False,
        L=math.inf, Lq=math.inf, W=math.inf, Wq=math.inf,
        P0=0.0, Pw=1.0, Pn=[],
    )

# P0 from the normalization sum.
head = sum(a ** n / math.factorial(n) for n in range(s))
tail = a ** s / (math.factorial(s) * (1.0 - rho))
P0 = 1.0 / (head + tail)

Pw = (a ** s / (math.factorial(s) * (1.0 - rho))) * P0 # Erlang-C
Lq = Pw * rho / (1.0 - rho)
Wq = Lq / lam if lam > 0 else 0.0
W = Wq + 1.0 / mu
L = lam * W

# State probabilities P(n in system).
Pn = []
for n in range(n_probs):
    if n < s:
        Pn.append(a ** n / math.factorial(n) * P0)
    else:
        Pn.append(a ** n / (math.factorial(s) * s ** (n - s)) * P0)

return QueueMetrics(
    model="M/M/S", lam=lam, mu=mu, servers=s, rho=rho, stable=True,
    L=L, Lq=Lq, W=W, Wq=Wq, P0=P0, Pw=Pw, Pn=Pn,
)

```

queuing/simulation.py

```

"""
queuing/simulation.py
Discrete-event simulator for an s-server FCFS queue (Paper 3 §3).

```

Purpose

1. Cross-check the closed-form M/M/1 / M/M/S formulas in `models.py`: with exponential inter-arrival and service times the simulated W , W_q , L , L_q must converge to the analytic values.
2. Drive the randomness study: swap the inter-arrival or service distribution (uniform / poisson, see `distributions.py`) -- which makes the system a G/G/s queue with no simple closed form -- and measure how the quantities shift away from the M/M baseline.

Method

Customers are generated in arrival order; each is dispatched FCFS to whichever of the s servers becomes free earliest (exact for FCFS). Per-customer sojourn and wait times are recorded. After discarding a warm-up prefix:

$$W = \text{mean sojourn time} \quad W_q = \text{mean queue wait}$$

$$L = (\text{sum of sojourn times}) / T \quad L_q = (\text{sum of queue waits}) / T$$

using the identity $\int_0^T N dt = \text{sum}(\text{sojourn times})$ over the window T .

```

"""
from __future__ import annotations

import random
import statistics as st
from dataclasses import dataclass

from .distributions import sampler

@dataclass
class SimResult:
    """Measured queueing quantities from one simulation run."""

    model: str
    servers: int
    lam: float
    mu: float
    rho: float
    arrival_dist: str
    service_dist: str
    n_measured: int
    L: float
    Lq: float
    W: float
    Wq: float
    server_utilization: float

```

```

def as_row(self) -> dict:
    return {
        "model": self.model,
        "servers": self.servers,
        "lambda": self.lam,
        "mu": self.mu,
        "rho": self.rho,
        "arrival_dist": self.arrival_dist,
        "service_dist": self.service_dist,
        "n_measured": self.n_measured,
        "L": self.L,
        "Lq": self.Lq,
        "W": self.W,
        "Wq": self.Wq,
        "server_utilization": self.server_utilization,
    }

def simulate_queue(
    lam: float,
    mu: float,
    *,
    servers: int = 1,
    n_arrivals: int = 100_000,
    warmup: int = 5_000,
    arrival_dist: str = "exponential",
    service_dist: str = "exponential",
    rng: random.Random | None = None,
) -> SimResult:
    """Simulate an s-server FCFS queue and return its measured quantities.

    Parameters
    -----
    lam, mu : arrival rate and per-server service rate.
    servers : number of identical parallel servers (s).
    n_arrivals : total customers to simulate (-> infinity as it grows).
    warmup : leading customers discarded before measuring steady
state.
    arrival_dist : distribution of inter-arrival times (mean held at
1/lam).
    service_dist : distribution of service times (mean held at
1/mu).
    rng : seeded random.Random for reproducibility.
    """
    rng = rng or random.Random()
    draw_gap = sampler(arrival_dist)
    draw_svc = sampler(service_dist)
    mean_gap = 1.0 / lam
    mean_svc = 1.0 / mu

    free_at = [0.0] * servers # when each server next becomes
free

```

```

clock = 0.0 # running arrival clock
busy_area = 0.0 # integral of (#busy servers) dt --
crude

sojourns: list[float] = [] # W_i for measured customers
waits: list[float] = [] # Wq_i for measured customers
first_arrival = last_departure = None

for i in range(n_arrivals):
    clock += draw_gap(mean_gap, rng)
    arrival = clock

    # Dispatch FCFS to the earliest-free server.
    srv = min(range(servers), key=lambda k: free_at[k])
    service_start = max(arrival, free_at[srv])
    service_time = draw_svc(mean_svc, rng)
    departure = service_start + service_time
    free_at[srv] = departure

    if i >= warmup:
        wq = service_start - arrival
        w = departure - arrival
        waits.append(wq)
        sojourns.append(w)
        busy_area += service_time
    if first_arrival is None:
        first_arrival = arrival
    last_departure = departure

n = len(sojourns)
rho = lam / (servers * mu)
if n == 0 or first_arrival is None:
    return SimResult(
        model=f"M/M/{servers}" if servers > 1 else "M/M/1",
        servers=servers, lam=lam, mu=mu, rho=rho,
        arrival_dist=arrival_dist, service_dist=service_dist,
        n_measured=0, L=0.0, Lq=0.0, W=0.0, Wq=0.0,
server_utilization=0.0,
    )

window = last_departure - first_arrival
return SimResult(
    model=f"M/M/{servers}" if servers > 1 else "M/M/1",
    servers=servers, lam=lam, mu=mu, rho=rho,
    arrival_dist=arrival_dist, service_dist=service_dist,
    n_measured=n,
    W=st.fmean(sojourns),
    Wq=st.fmean(waits),
    L=sum(sojourns) / window if window > 0 else 0.0,
    Lq=sum(waits) / window if window > 0 else 0.0,
    server_utilization=busy_area / (servers * window) if window > 0

```



```

else
    )

```

0.0,

Appendix B — Drivers and plot scripts

drivers/_p3_common.py

```

"""
drivers/_p3_common.py
Shared plumbing for the Paper 3 drivers (run_mlfq, run_queuing, plot_*).

Handles three things every driver needs:
1. sys.path wiring so `import mlfq`, `import queuing` and `from common
import
plots` all resolve regardless of the (hyphenated) directory names.
2. loading config/experiments.yaml.
3. an append-only CSV writer whose header is materialised from the first
row.
"""
from __future__ import annotations

import csv
import sys
from pathlib import Path

# paper3-scheduling/ -> for `import mlfq`, `import queuing`
PAPER3_DIR = Path(__file__).resolve().parents[1]
# repo root -> for `from common import plots`
ROOT = Path(__file__).resolve().parents[2]
for _p in (str(PAPER3_DIR), str(ROOT)):
    if _p not in sys.path:
        sys.path.insert(0, _p)

import yaml # noqa: E402

RESULTS_DIR = PAPER3_DIR / "results"
FIGURES_DIR = PAPER3_DIR / "figures"

def load_cfg() -> dict:
    """Parse config/experiments.yaml from the repo root."""
    return yaml.safe_load((ROOT / "config" /
"experiments.yaml").read_text())

def paper3_cfg() -> dict:
    """Just the `paper3:` block of the config."""
    return load_cfg()["paper3"]

def global_seeds() -> list[int]:
    """The shared RNG seed list (global.seeds in the config)."""
    return load_cfg()["global"]["seeds"]

```

```

class CsvSink:
    """Append-only CSV writer; header taken from the first row's keys.

    Later rows are written with extrasaction='ignore', so a row with a
    slightly different field set does not abort the run.
    """

    def __init__(self, out_path: Path):
        self.path = Path(out_path)
        self.path.parent.mkdir(parents=True, exist_ok=True)
        self._f = None
        self._writer: csv.DictWriter | None = None

    def write(self, row: dict) -> None:
        if self._writer is None:
            is_new = not (self.path.exists() and self.path.stat().st_size
                           > 0)
            self._f = self.path.open("a", newline="")
            self._writer = csv.DictWriter(
                self._f, fieldnames=list(row.keys()),
                extrasaction="ignore"
            )
            if is_new:
                self._writer.writeheader()
                self._writer.writerow(row)

    def close(self) -> None:
        if self._f:
            self._f.close()

    def __enter__(self) -> "CsvSink":
        return self

    def __exit__(self, *exc) -> None:
        self.close()

```

drivers/run_mlfq.py

```

"""
drivers/run_mlfq.py
Monte Carlo grid search over the MLFQ scheduler parameters (Paper 3 §2,
§6).

```

For every valid (Q1, Q2, L1, L2[, T]) combination the driver runs `trials` independent simulations -- each with a freshly generated random workload -- and writes one CSV row holding the trial-averaged metrics. After the sweep it prints the optimal configuration for each of the four objectives.

Usage

```

# fast smoke run on a coarse grid

```

```

python paper3-scheduling/drivers/run_mlfq.py --variant A

# full grid from config/experiments.yaml (long -- reduce first if
needed)
python paper3-scheduling/drivers/run_mlfq.py --variant A --full --
trials 300

Output: paper3-scheduling/results/mlfq_<variant>.csv

Note on cost: the full config grid (Q1 x Q2 x L1 x L2 [x T]) times 300
trials
times N=1000 processes is large -- exactly as Paper 2's sweep was reduced
to
fit the ARM VM, the MLFQ grid should be trimmed for the real run. The
coarse
default grid below is the smoke-test subset.
"""
from __future__ import annotations

import argparse
import itertools
import multiprocessing as mp
import os
import random
import sys
import time
from functools import partial

from _p3_common import RESULTS_DIR, CsvSink, global_seeds, paper3_cfg

from mlfq import MLFQConfig, generate_processes, simulate, summarize
from mlfq.metrics import mean_of

# Coarse default grid -- a smoke-test subset of the full config grid.
COARSE = {
    "Q1": [2, 8, 20],
    "Q2": [8, 24, 48],
    "L1": [20, 40],
    "L2": [30, 50],
    "T": [0, 50, 100],
}

OBJECTIVES = [
    ("throughput", "max"),
    ("turnaround_mean", "min"),
    ("waiting_mean", "min"),
    ("response_mean", "min"),
]

def build_grid(cfg: dict, variant: str, full: bool) -> list[MLFQConfig]:
    """Materialise every valid MLFQConfig for the requested grid."""

```

```

        cycle = mlfq["total_level_time_ms"]
        if full:
            q1s, q2s = mlfq["Q1_grid"], mlfq["Q2_grid"]
            l1s, l2s = mlfq["L1_grid"], mlfq["L2_grid"]
            ts = mlfq["T_grid"]
        else:
            q1s, q2s = COARSE["Q1"], COARSE["Q2"]
            l1s, l2s = COARSE["L1"], COARSE["L2"]
            ts = COARSE["T"]

        t_values = ts if variant == "B" else [0]
        grid: list[MLFQConfig] = []
        for q1, q2, l1, l2, t in itertools.product(q1s, q2s, l1s, l2s,
t_values):
            c = MLFQConfig(q1=q1, q2=q2, l1=l1, l2=l2, cycle=cycle,
variant=variant, t_split=t)
            if c.is_valid():
                grid.append(c)
        return grid

def evaluate(config: MLFQConfig, n: int, m: float, trials: int,
seeds: list[int], arrival_dist: str, burst_range: tuple[int,
int])
    ) -> dict:
    """Run `trials` simulations of `config`; return trial-averaged
metrics.

    The per-trial seed is a deterministic function of the trial index, so
the
    averaged result is identical whether configs run serially or in
parallel.

    """
    runs = []
    for t in range(trials):
        seed = seeds[t % len(seeds)] * 100_003 + t # spread seeds across
trials
        rng = random.Random(seed)
        procs = generate_processes(n, m, burst_range=burst_range,
arrival_dist=arrival_dist, rng=rng)
        sim = simulate(procs, config)
        runs.append(summarize(sim.procs, sim.context_switches))
    return {k: mean_of(runs, k) for k in runs[0]}

def _worker(config: MLFQConfig, params: dict) -> tuple[MLFQConfig, dict]:
    """multiprocessing.Pool worker -- evaluate one config (module-level so
it
    is picklable). Each config is independent, so the grid parallelises
cleanly across cores."""
    return config, evaluate(

```

```

        config, params["n"], params["m"], params["trials"],
        params["seeds"], params["arrival_dist"], params["burst_range"],
    )

def main(argv: list[str] | None = None) -> int:
    cfg = paper3_cfg()
    mlfq = cfg["mlfq"]

    ap = argparse.ArgumentParser(description="MLFQ Monte Carlo grid
search.")
    ap.add_argument("--variant", choices=["A", "B"], default="A")
    ap.add_argument("--full", action="store_true",
help="use the full config grid instead of the coarse
one")
    ap.add_argument("--trials", type=int, default=10,
help="Monte Carlo trials per config (config: 300)")
    ap.add_argument("--n", type=int, default=200, help="processes per
trial")
    ap.add_argument("--m", type=float, default=100.0, help="arrival window
width")
    ap.add_argument("--arrival-dist", default="uniform",
choices=["uniform", "exponential", "poisson"])
    ap.add_argument("--out", default=None, help="output CSV path
override")
    ap.add_argument("--jobs", type=int, default=0,
help="parallel worker processes (0 = all CPU cores)")
    args = ap.parse_args(argv)

    burst_range = tuple(mlfq["burst_range"])
    seeds = global_seeds()
    grid = build_grid(cfg, args.variant, args.full)
    out_path = (RESULTS_DIR / f"mlfq_{args.variant}.csv"
if args.out is None else args.out)
    jobs = args.jobs if args.jobs > 0 else (os.cpu_count() or 1)

    print(f"[run_mlfq] variant={args.variant} configs={len(grid)} "
f"trials={args.trials} N={args.n} M={args.m} jobs={jobs} ->
{out_path}",
file=sys.stderr)

    params = {
        "n": args.n, "m": args.m, "trials": args.trials, "seeds": seeds,
        "arrival_dist": args.arrival_dist, "burst_range": burst_range,
    }
    best: dict[str, tuple[float, dict]] = {}
    started = time.time()

    # The grid parallelises across cores -- one process per config.
    imap_un-
    # ordered streams results back as workers finish so the CSV and
    progress

```

```

# line update live; trial seeding is index-based so results are
identical

# to a serial run.
with CsvSink(out_path) as sink, mp.Pool(jobs) as pool:
    stream = pool.imap_unordered(partial(_worker, params=params),
grid)

    for i, (config, metrics) in enumerate(stream, 1):
        row = {
            "variant": config.variant,
            "Q1": config.q1, "Q2": config.q2,
            "L1": config.l1, "L2": config.l2, "L3": config.l3,
            "T": config.t_split,
            "N": args.n, "M": args.m,
            "trials": args.trials,
            "arrival_dist": args.arrival_dist,
            **metrics,
        }
        sink.write(row)
        for key, direction in OBJECTIVES:
            val = row[key]
            if key not in best or (
                (direction == "max" and val > best[key][0])
                or (direction == "min" and val < best[key][0])
            ):
                best[key] = (val, row)
        if i % 10 == 0 or i == len(grid):
            print(f" [{i}/{len(grid)}] {time.time()-started:.1f}s",
                  file=sys.stderr)

    print(f"\n[run_mlfq] done in {time.time()-started:.1f}s. Optimal
configs:")

    for key, direction in OBJECTIVES:
        val, row = best[key]
    print(f" {direction} {key:16s} = {val:12.4f} at "
          f"Q1={row['Q1']} Q2={row['Q2']} L1={row['L1']}
L2={row['L2']}")
    + (f" T={row['T']}" if args.variant == "B" else "")
    return 0

if __name__ == "__main__":
    raise SystemExit(main())

drivers/run_queueing.py
"""
drivers/run_queueing.py
M/M/1 and M/M/S queueing sweeps (Paper 3 §3).

Two things, both written to paper3-scheduling/results/:

1. mq_analytic.csv -- closed-form M/M/1 and M/M/S quantities swept over
the
constant (N, M, R) grid and the server-count grid. The project's "N,

```

```

M,
    constant over time" case: lambda = N/M, mu = R.

2. mq_randomness.csv (with --simulate) -- the randomness study: the same
   operating points fed to the discrete-event simulator under each of
the
   three distributions (exponential / uniform / poisson). The
exponential
   should match the analytic value; the others quantify the shape
effect.

Usage
-----

python paper3-scheduling/drivers/run_queuing.py
python paper3-scheduling/drivers/run_queuing.py --simulate --arrivals
50000
"""
from __future__ import annotations

import argparse
import random
import sys
import time

from _p3_common import RESULTS_DIR, CsvSink, global_seeds, paper3_cfg
from queuing import mml_metrics, mms_metrics, simulate_queue

def analytic_sweep(cfg: dict, sink: CsvSink) -> int:
    """Closed-form M/M/1 + M/M/S over the constant (N, M, R) grid."""
    q = cfg["queuing"]
    grid = q["constant_grid"]
    rows = 0
    for n in grid["N"]:
        for m in grid["M"]:
            for r in grid["R"]:
                lam = n / m # arrival rate
                mu = float(r) # per-server service rate
                # M/M/1
                row = mml_metrics(lam, mu).as_row()
                row.update(N=n, M=m, R=r)
                sink.write(row)
                rows += 1
            # M/M/S for every server count in the grid
            for s in q["S_servers"]:
                row = mms_metrics(lam, mu, s).as_row()
                row.update(N=n, M=m, R=r)
                sink.write(row)
                rows += 1
    return rows

```

```

def randomness_sweep(cfg: dict, sink: CsvSink, n_arrivals: int,
                    warmup: int, seeds: list[int]) -> int:
    """Simulate each operating point under all three distributions."""
    q = cfg["queuing"]
    grid = q["constant_grid"]
    dists = q["distributions"]
    rows = 0
    for n in grid["N"]:
        for m in grid["M"]:
            for r in grid["R"]:
                lam = n / m
                mu = float(r)
                if lam / mu >= 1.0:
                    continue # unstable -- simulator would not converge
                for s in [1] + [x for x in q["S_servers"] if x > 1]:
                    if lam / (s * mu) >= 1.0:
                        continue
                    for dist in dists:
                        res = simulate_queue(
                            lam, mu, servers=s,
                            n_arrivals=n_arrivals, warmup=warmup,
                            arrival_dist=dist, service_dist=dist,
                            rng=random.Random(seeds[0] * 911 + s),
                        )
                        row = res.as_row()
                        row.update(N=n, M=m, R=r)
                        sink.write(row)
                        rows += 1
    return rows

def main(argv: list[str] | None = None) -> int:
    cfg = paper3_cfg()
    q = cfg["queuing"]

    ap = argparse.ArgumentParser(description="M/M/1 and M/M/S queueing sweeps.")
    ap.add_argument("--simulate", action="store_true",
                    help="also run the discrete-event randomness study")
    ap.add_argument("--arrivals", type=int,
                    default=q["steady_state_arrivals"],
                    help="customers per simulation (config default)")
    ap.add_argument("--warmup", type=int, default=q["warmup_arrivals"])
    args = ap.parse_args(argv)

    seeds = global_seeds()
    started = time.time()

    with CsvSink(RESULTS_DIR / "mq_analytic.csv") as sink:
        n = analytic_sweep(cfg, sink)
    print(f"[run_queuing] analytic: {n} rows -> results/mq_analytic.csv ")

```



```

        f"({time.time()-started:.1f}s)", file=sys.stderr)

        if args.simulate:
            with CsvSink(RESULTS_DIR / "mq_randomness.csv") as sink:
                n = randomness_sweep(cfg, sink, args.arrivals, args.warmup,
seeds)

                print(f"[run_queueing] randomness study: {n} rows -> "
                    f"results/mq_randomness.csv ({time.time()-started:.1f}s)",
                        file=sys.stderr)

            return 0

if __name__ == "__main__":
    raise SystemExit(main())

drivers/plot_mlfq.py
"""
drivers/plot_mlfq.py
Visualise the MLFQ grid search (Paper 3 §7): heatmaps + 2D + 3D surfaces.

Reads paper3-scheduling/results/mlfq_<variant>.csv (produced by
run_mlfq.py)
and writes figures to paper3-scheduling/figures/mlfq/.

For each metric it draws a Q1 x Q2 heatmap and a 3D performance surface,
taking
the (L1, L2) slice that is optimal for that metric so the figure shows the
best
achievable behaviour. A line plot of each metric vs Q1 is also produced.

Usage
-----

python paper3-scheduling/drivers/plot_mlfq.py --variant A

"""
from __future__ import annotations

import argparse
import csv
import sys
from collections import defaultdict

from _p3_common import FIGURES_DIR, RESULTS_DIR

from common.plots import heatmap, line_with_errorbars, save, surface3d
import numpy as np

# metric -> (human label, optimization direction)
METRICS = {
    "throughput": ("Throughput (jobs/time-unit)", "max"),
    "turnaround_mean": ("Mean turnaround time", "min"),
    "waiting_mean": ("Mean waiting time", "min"),
    "response_mean": ("Mean response time", "min"),

```

```
}
```

```
def load_rows(path) -> list[dict]:
    with open(path) as f:
        rows = list(csv.DictReader(f))
        for r in rows:
            for k, v in list(r.items()):
                try:
                    r[k] = float(v)
                except (ValueError, TypeError):
                    pass
        return rows

def best_slice(rows: list[dict], metric: str, direction: str) -> tuple[float, float]:
    """Return the (L1, L2) pair whose best cell is globally optimal."""
    pick = max if direction == "max" else min
    row = pick(rows, key=lambda r: r[metric])
    return row["L1"], row["L2"]

def grid_matrix(rows: list[dict], metric: str, l1: float, l2: float):
    """Build a Q1 x Q2 value matrix for the fixed (L1, L2) slice."""
    sliced = [r for r in rows if r["L1"] == l1 and r["L2"] == l2]
    q1s = sorted({r["Q1"] for r in sliced})
    q2s = sorted({r["Q2"] for r in sliced})
    lookup = {(r["Q1"], r["Q2"]): r[metric] for r in sliced}
    M = np.array([[lookup.get((q1, q2), np.nan) for q1 in q1s] for q2 in q2s])
    return M, q1s, q2s

def main(argv: list[str] | None = None) -> int:
    ap = argparse.ArgumentParser(description="Plot the MLFQ grid search.")
    ap.add_argument("--variant", choices=["A", "B"], default="A")
    args = ap.parse_args(argv)

    src = RESULTS_DIR / f"mlfq_{args.variant}.csv"
    if not src.exists():
        print(f"[plot_mlfq] missing {src}; run run_mlfq.py first",
              file=sys.stderr)
    return 1

    rows = load_rows(src)
    out = FIGURES_DIR / "mlfq"
    print(f"[plot_mlfq] {len(rows)} rows from {src.name} -> {out}/",
          file=sys.stderr)

    for metric, (label, direction) in METRICS.items():
        l1, l2 = best_slice(rows, metric, direction)
```

```

M, q1s, q2s = grid_matrix(rows, metric, l1, l2)

# Heatmap over the Q1 x Q2 grid.
fig, _ = heatmap(
    M, xticks=q1s, yticks=q2s,
    xlabel="Q1 (level-0 quantum)", ylabel="Q2 (level-1 quantum)",
    title=f"{label} (variant {args.variant}, L1={l1:.0f},
L2={l2:.0f})",
    cbar_label=label, annotate=True, annot_fmt="{:.1f}",
)
save(fig, out / f"heatmap_{metric}_{args.variant}.png")

# 3D performance surface.
X, Y = np.meshgrid(q1s, q2s)
fig, _ = surface3d(
    X, Y, M,
    xlabel="Q1", ylabel="Q2", zlabel=label,
    title=f"{label} surface (variant {args.variant})",
)
save(fig, out / f"surface_{metric}_{args.variant}.png")

# 2D line: metric vs Q1, one line per Q2 (best slice).
sliced = [r for r in rows if r["L1"] == l1 and r["L2"] == l2]
by_q2 = defaultdict(list)
for r in sliced:
    by_q2[r["Q2"]].append((r["Q1"], r[metric]))
fig, ax = None
for q2 in sorted(by_q2):
    pts = sorted(by_q2[q2])
    xs = [p[0] for p in pts]
    ys = [p[1] for p in pts]
    fig, ax = line_with_errorbars(
        xs, ys, label=f"Q2={q2:.0f}", ax=ax,
        xlabel="Q1 (level-0 quantum)", ylabel=label,
        title=f"{label} vs Q1 (variant {args.variant})",
    )
if fig is not None:
    save(fig, out / f"line_{metric}_{args.variant}.png")

print("[plot_mlfq] done.", file=sys.stderr)
return 0

```

```

if __name__ == "__main__":
    raise SystemExit(main())

```

drivers/plot_queueing.py

"""

drivers/plot_queueing.py

Visualise the queueing sweeps (Paper 3 \$7, \$9).

Reads paper3-scheduling/results/mq_analytic.csv (and mq_randomness.csv if it

exists) and writes figures to paper3-scheduling/figures/queuing/:

- L, Lq, W, Wq and rho versus the server rate R, per model.
- the same quantities versus the arrival load (N/M).
- if the randomness study ran: a bar comparison of W and Wq across the exponential / uniform / poisson distributions.

Usage

```
python paper3-scheduling/drivers/plot_queuing.py

"""
from __future__ import annotations

import csv
import sys
from collections import defaultdict

from _p3_common import FIGURES_DIR, RESULTS_DIR

from common.plots import bar_compare, line_with_errorbars, save

def load_rows(path) -> list[dict]:
    with open(path) as f:
        rows = list(csv.DictReader(f))
        for r in rows:
            for k, v in list(r.items()):
                try:
                    r[k] = float(v)
                except (ValueError, TypeError):
                    pass
        return rows

def plot_vs_R(rows: list[dict], out) -> None:
    """Quantities vs server rate R, one line per model (M/M/1, M/M/S by
s)."""
    # Pick a representative stable (N, M) operating point.
    stable = [r for r in rows if r.get("stable") in (True, "True", 1.0)]
    if not stable:
        print("[plot_queuing] no stable rows to plot vs R",
file=sys.stderr)
    return
    n0 = sorted({r["N"] for r in stable})[len(set(r["N"] for r in stable))
// 2]
    m0 = sorted({r["M"] for r in stable})[len(set(r["M"] for r in stable))
// 2]
    sel = [r for r in stable if r["N"] == n0 and r["M"] == m0]

    for metric in ("L", "Lq", "W", "Wq"):
        by_model = defaultdict(list)
        for r in sel:
```

```

        key = f"{r['model']}" s={int(r['servers'])}"
        by_model[key].append((r["R"], r[metric]))
        fig = ax = None
        for key in sorted(by_model):
            pts = sorted(by_model[key])
            fig, ax = line_with_errorbars(
                [p[0] for p in pts], [p[1] for p in pts],
                label=key, ax=ax,
                xlabel="Server rate R", ylabel=metric,
                title=f"{metric} vs R (N={n0:.0f}, M={m0:.0f})",
            )
            if fig is not None:
                save(fig, out / f"{metric}_vs_R.png")

def plot_vs_load(rows: list[dict], out) -> None:
    """Mean number in system L vs offered load lambda = N/M, per server
    count."""
    stable = [r for r in rows if r.get("stable") in (True, "True", 1.0)]
    if not stable:
        return
    r0 = sorted({r["R"] for r in stable})[len(set(r["R"] for r in stable))
// 2]
    sel = [r for r in stable if r["R"] == r0]
    by_s = defaultdict(list)
    for r in sel:
        by_s[int(r["servers"])].append((r["lambda"], r["L"]))
    fig = ax = None
    for s in sorted(by_s):
        pts = sorted(by_s[s])
        fig, ax = line_with_errorbars(
            [p[0] for p in pts], [p[1] for p in pts],
            label=f"s={s}", ax=ax,
            xlabel="Arrival rate lambda = N/M", ylabel="L (jobs in
system)",
            title=f"L vs arrival load (R={r0:.0f})",
        )
        if fig is not None:
            save(fig, out / "L_vs_load.png")

def plot_randomness(rows: list[dict], out) -> None:
    """Bar comparison of W / Wq across the three distributions."""
    by_dist = defaultdict(lambda: defaultdict(list))
    for r in rows:
        by_dist[r["service_dist"]]["W"].append(r["W"])
        by_dist[r["service_dist"]]["Wq"].append(r["Wq"])
    dists = sorted(by_dist)
    if not dists:
        return
    series = {
        "W (time in system)": [

```

```

        sum(by_dist[d]["W"])) / len(by_dist[d]["W"])) for d in dists
    ],
    "Wq (queue wait)": [
        sum(by_dist[d]["Wq"])) / len(by_dist[d]["Wq"])) for d in dists
    ],
}

fig, _ = bar_compare(
    dists, series,
    xlabel="Distribution", ylabel="Mean time",
    title="Randomness study: queueing delay by distribution",
)

save(fig, out / "randomness_W_Wq.png")

def main(argv: list[str] | None = None) -> int:
    out = FIGURES_DIR / "queuing"
    analytic = RESULTS_DIR / "mq_analytic.csv"
    if not analytic.exists():
        print(f"[plot_queuing] missing {analytic}; run run_queuing.py
first",
            file=sys.stderr)
        return 1

    rows = load_rows(analytic)
    print(f"[plot_queuing] {len(rows)} analytic rows -> {out}/",
file=sys.stderr)

    plot_vs_R(rows, out)
    plot_vs_load(rows, out)

    randomness = RESULTS_DIR / "mq_randomness.csv"
    if randomness.exists():
        plot_randomness(load_rows(randomness), out)
        print("[plot_queuing] randomness figure written.",
file=sys.stderr)

    print("[plot_queuing] done.", file=sys.stderr)
    return 0

if __name__ == "__main__":
    raise SystemExit(main())

```

Appendix C — Raw runs (samples)

results/mlfq_A.csv (first 25 lines)

```

variant,Q1,Q2,L1,L2,L3,T,N,M,trials,arrival_dist,n_completed,throughput,turnaround_mean,waiting_mean,response_mean,turnaround_p95,waiting_p95,makespan,context_switches
A,2,16,20,30,50,0,500,100.0,100,uniform,500.0,0.001979054402382489,168934.0009447436,168428.3765447436,2405.5501447435977,251571.60417109533,250634.832504066,252812.41298856243,6708.44
A,2,8,20,30,50,0,500,100.0,100,uniform,500.0,0.001979054402382489,169620.0972047436,169114.47280474362,2409.3281447435975,251594.80857163857,250658.

```

0547260223,252812.41298856243,6590.07
A,2,16,40,30,30,0,500,100.0,100,uniform,500.0,0.001979054402382489,169694.4171047436,169188.7927047436,1167.6901447435976,251845.8367484426,250903.84968667914,252812.41298856243,9864.05
A,2,8,20,50,30,0,500,100.0,100,uniform,500.0,0.001979054402382489,170042.9842447436,169537.35984474362,2409.3281447435975,251866.01278478027,250919.03581008196,252812.41298856243,9819.81
A,2,8,40,30,30,0,500,100.0,100,uniform,500.0,0.001979054402382489,169967.42470474358,169461.8003047436,1169.3281447435975,251865.49352766256,250919.5120479608,252812.41298856243,9873.75
A,2,16,20,50,30,0,500,100.0,100,uniform,500.0,0.001979054402382489,169951.1438247436,169445.51942474357,2409.3281447435975,251839.41356260885,250897.09691819747,252812.41298856243,9759.78
A,2,32,20,30,50,0,500,100.0,100,uniform,500.0,0.001979054402382489,169997.2696047436,169491.6452047436,1184.3281447435975,251881.6071269737,250938.1082532605,252812.41298856243,9375.89
A,2,32,20,50,30,0,500,100.0,100,uniform,500.0,0.001979054402382489,169662.09738474357,169156.4729847436,2406.7917447435975,251805.6749587892,250861.63561140854,252812.41298856243,9530.04
A,2,32,40,30,30,0,500,100.0,100,uniform,500.0,0.001979054402382489,170023.6495047436,169518.0251047436,809.3281447435974,251887.32517938744,250935.80109384566,252812.41298856243,9376.22
A,6,8,20,30,50,0,500,100.0,100,uniform,500.0,0.001979054402382489,169631.1294247436,169125.50502474362,5804.000804743597,251618.73923271158,250690.21342678825,252812.41298856243,6561.73
A,2,48,20,50,30,0,500,100.0,100,uniform,500.0,0.001979054402382489,169543.81572474362,169038.1913247436,2405.7295447435977,251769.25255020632,250828.96626946202,252812.41298856243,9520.45
A,2,48,20,30,50,0,500,100.0,100,uniform,500.0,0.001979054402382489,169997.2696047436,169491.6452047436,1184.3281447435975,251881.6071269737,250938.1082532605,252812.41298856243,9375.89
A,2,48,40,30,30,0,500,100.0,100,uniform,500.0,0.001979054402382489,170023.6495047436,169518.0251047436,809.3281447435974,251887.32517938744,250935.80109384566,252812.41298856243,9376.22
A,6,8,20,50,30,0,500,100.0,100,uniform,500.0,0.001979054402382489,170000.59328474358,169494.9688847436,4561.083484743597,251877.06671040773,250930.51218460526,252812.41298856243,9827.81
A,6,16,20,30,50,0,500,100.0,100,uniform,500.0,0.001979054402382489,168850.87710474362,168345.2527047436,6175.834944743598,251506.25306104164,250575.98328827842,252812.41298856243,6824.26
A,6,8,40,30,30,0,500,100.0,100,uniform,500.0,0.001979054402382489,169975.4677447436,169469.8433447436,3498.3943447435972,251897.52904075675,250951.13710561066,252812.41298856243,9876.15
A,2,16,40,50,10,0,500,100.0,100,uniform,500.0,0.001979054402382489,170147.9155047436,169642.2911047436,1169.3281447435975,252068.52535954886,251121.26158096522,252812.41298856243,25956.76
A,2,8,40,50,10,0,500,100.0,100,uniform,500.0,0.001979054402382489,170184.5543647436,169678.92996474358,1169.3281447435975,252095.32357386444,251146.66527600866,252812.41298856243,26296.46
A,6,16,20,50,30,0,500,100.0,100,uniform,500.0,0.001979054402382489,169960.18698474357,169454.5625847436,6039.998484743597,251876.27376226065,250932.26903175365,252812.41298856243,9692.53

A,6,16,40,30,30,0,500,100.0,100,uniform,500.0,0.001979054402382489,169693.4183847436,169187.7939847436,3496.6450047435974,251810.8118496593,250866.0155467423,252812.41298856243,9867.55
A,2,32,40,50,10,0,500,100.0,100,uniform,500.0,0.001979054402382489,170048.66170474357,169543.0373047436,1168.6725447435974,252015.46969422494,251068.69349229764,252812.41298856243,24667.5
A,6,32,20,30,50,0,500,100.0,100,uniform,500.0,0.001979054402382489,169984.7953047436,169479.1709047436,3072.9837847435974,251884.84778885831,250939.33727539127,252812.41298856243,9464.6
A,2,48,40,50,10,0,500,100.0,100,uniform,500.0,0.001979054402382489,170039.5166047436,169533.89220474358,1168.3089447435973,252072.33245071542,251122.62984023697,252812.41298856243,24613.08
A,6,32,20,50,30,0,500,100.0,100,uniform,500.0,0.001979054402382489,169641.5681647436,169135.94376474357,6177.239784743597,251786.03223369774,250845.9051838238,252812.41298856243,9618.24

results/mlfq_B.csv (first 25 lines)

variant,Q1,Q2,L1,L2,L3,T,N,M,triangles,arrival_dist,n_completed,throughput,turnaround_mean,waiting_mean,response_mean,turnaround_p95,waiting_p95,makespan,context_switches
B,2,8,20,30,50,0,500,100.0,100,uniform,500.0,0.001979054402382489,169620.0972047436,169114.47280474362,2409.3281447435975,251594.80857163857,250658.0547260223,252812.41298856243,6590.07
B,2,8,20,50,30,0,500,100.0,100,uniform,500.0,0.001979054402382489,170042.9842447436,169537.35984474362,2409.3281447435975,251866.01278478027,250919.03581008196,252812.41298856243,9819.81
B,2,8,40,30,30,0,500,100.0,100,uniform,500.0,0.001979054402382489,169967.42470474358,169461.8003047436,1169.3281447435975,251865.49352766256,250919.5120479608,252812.41298856243,9873.75
B,2,16,20,30,50,0,500,100.0,100,uniform,500.0,0.001979054402382489,168934.0009447436,168428.3765447436,2405.5501447435977,251571.60417109533,250634.832504066,252812.41298856243,6708.44
B,2,8,40,50,10,0,500,100.0,100,uniform,500.0,0.001979054402382489,170184.5543647436,169678.92996474358,1169.3281447435975,252095.32357386444,251146.66527600866,252812.41298856243,26296.46
B,2,8,20,30,50,100,500,100.0,100,uniform,500.0,0.001979054402382489,88200.1068047436,87694.4824047436,2409.3281447435975,228703.05022165028,227754.22022165026,252812.41298856243,6590.62
B,2,16,20,50,30,0,500,100.0,100,uniform,500.0,0.001979054402382489,169951.1438247436,169445.51942474357,2409.3281447435975,251839.41356260885,250897.09691819747,252812.41298856243,9759.78
B,2,16,20,30,50,50,500,100.0,100,uniform,500.0,0.001979054402382489,100645.0450447436,100139.42064474359,2405.5501447435977,231238.69290267228,230390.06269215565,252812.41298856243,11442.37
B,2,8,20,30,50,50,500,100.0,100,uniform,500.0,0.001979054402382489,96703.0398047436,96197.41540474359,2409.3281447435975,230830.466793898,229984.05568772726,252812.41298856243,11517.34
B,2,16,20,30,50,100,500,100.0,100,uniform,500.0,0.001979054402382489,92508.60850474359,92002.9841047436,2405.5501447435977,229117.1567886653,228170.82678866532,252812.41298856243,6709.14
B,2,16,40,30,30,0,500,100.0,100,uniform,500.0,0.001979054402382489,169694.4171047436,169188.7927047436,1167.6901447435976,251845.8367484426,250903.84968667914,252812.41298856243,9864.05

B,2,8,40,30,30,100,500,100.0,100,uniform,500.0,0.001979054402382489,88178.1475247436,87672.52312474359,1169.3281447435975,228702.4874039588,227754.0774039588,252812.41298856243,9874.84
B,2,8,20,50,30,100,500,100.0,100,uniform,500.0,0.001979054402382489,87800.00768474358,87294.3832847436,2409.3281447435975,228658.88032519005,227710.01032519006,252812.41298856243,9820.65
B,2,8,40,30,30,50,500,100.0,100,uniform,500.0,0.001979054402382489,96619.36120474359,96113.73680474359,1169.3281447435975,230862.9394941439,230020.02083594276,252812.41298856243,18085.41
B,2,8,20,50,30,50,500,100.0,100,uniform,500.0,0.001979054402382489,96271.27800474361,95765.6536047436,2409.3281447435975,230809.16891760723,229959.12627304826,252812.41298856243,18058.3
B,2,16,40,50,10,0,500,100.0,100,uniform,500.0,0.001979054402382489,170147.9155047436,169642.2911047436,1169.3281447435975,252068.52535954886,251121.26158096522,252812.41298856243,25956.76
B,2,32,20,30,50,0,500,100.0,100,uniform,500.0,0.001979054402382489,169997.2696047436,169491.6452047436,1184.3281447435975,251881.6071269737,250938.1082532605,252812.41298856243,9375.89
B,2,32,20,30,50,50,500,100.0,100,uniform,500.0,0.001979054402382489,169997.2696047436,169491.6452047436,1184.3281447435975,251881.6071269737,250938.1082532605,252812.41298856243,9375.89
B,2,32,20,30,50,100,500,100.0,100,uniform,500.0,0.001979054402382489,169997.2696047436,169491.6452047436,1184.3281447435975,251881.6071269737,250938.1082532605,252812.41298856243,9375.89
B,2,32,20,50,30,0,500,100.0,100,uniform,500.0,0.001979054402382489,169662.09738474357,169156.4729847436,2406.7917447435975,251805.6749587892,250861.63561140854,252812.41298856243,9530.04
B,2,16,40,30,30,100,500,100.0,100,uniform,500.0,0.001979054402382489,92189.8786847436,91684.2542847436,1167.6901447435976,229098.16811191797,228151.83811191798,252812.41298856243,9864.64
B,2,16,20,50,30,100,500,100.0,100,uniform,500.0,0.001979054402382489,89599.01006474359,89093.38566474359,2409.3281447435975,228870.11428245355,227921.31428245353,252812.41298856243,9760.47
B,2,32,40,30,30,0,500,100.0,100,uniform,500.0,0.001979054402382489,170023.6495047436,169518.0251047436,809.3281447435974,251887.32517938744,250935.80109384566,252812.41298856243,9376.22
B,2,16,40,30,30,50,500,100.0,100,uniform,500.0,0.001979054402382489,100348.6170247436,99842.9926247436,1167.6901447435976,231251.75235336518,230404.19904591914,252812.41298856243,17752.98

results/mq_analytic.csv (first 25 lines)

model,lambda,mu,servers,rho,stable,L,Lq,W,Wq,P0,Pwait,N,M,R
M/M/1,2.0,1.0,1,2.0,False,inf,inf,inf,inf,0.0,1.0,100,50,1
M/M/S,2.0,1.0,1,2.0,False,inf,inf,inf,inf,0.0,1.0,100,50,1
M/M/S,2.0,1.0,2,1.0,False,inf,inf,inf,inf,0.0,1.0,100,50,1
M/M/S,2.0,1.0,4,0.5,True,2.1739130434782608,0.17391304347826086,1.0869565217391304,0.08695652173913043,0.13043478260869565,0.17391304347826086,100,50,1
M/M/S,2.0,1.0,8,0.25,True,2.000381879803332,0.00038187980333190133,1.000190939901666,0.00019093990166595067,0.13532865530574253,0.001145639409995704,100,50,1
M/M/1,2.0,5.0,1,0.4,True,0.6666666666666667,0.2666666666666667,0.3333333333333333,0.13333333333333333,0.6,0.4,100,50,5

M/M/S,2.0,5.0,1,0.4,True,0.6666666666666667,0.2666666666666667,0.3333333333333333,0.1333333333333333,0.6,0.4,100,50,5
M/M/S,2.0,5.0,2,0.2,True,0.4166666666666667,0.0166666666666667,0.2083333333333333,0.0083333333333333,0.6666666666666666,0.0666666666666668,100,50,5
M/M/S,2.0,5.0,4,0.1,True,0.40008827099194527,8.827099194527201e-05,0.20004413549597264,4.4135495972636005e-05,0.6703078450844091,0.000794438927507448,100,50,5
M/M/S,2.0,5.0,8,0.05,True,0.4000000006036211,6.036210940448153e-10,0.2000000030181056,3.0181054702240765e-10,0.6703200459893177,1.146880078685149e-08,100,50,5
M/M/1,2.0,10.0,1,0.2,True,0.25,0.05000000000000001,0.125,0.025,0.8,0.2,100,50,10
M/M/S,2.0,10.0,1,0.2,True,0.25,0.05000000000000001,0.125,0.025,0.8,0.2,100,50,10
M/M/S,2.0,10.0,2,0.1,True,0.20202020202020204,0.0020202020202021,0.10101010101010102,0.00101010101010105,0.8181818181818182,0.018181818181818188,100,50,10
M/M/S,2.0,10.0,4,0.05,True,0.2000030239344411,3.023934441101318e-06,0.10000151196722055,1.511967220550659e-06,0.8187302499281814,5.745475438092503e-05,100,50,10
M/M/S,2.0,10.0,8,0.025,True,0.20000000000136708,1.367071793749486e-12,0.1000000000068354,6.83535896874743e-13,0.8187307530778556,5.3315799956229947e-11,100,50,10
M/M/1,1.0,1.0,1,1.0,False,inf,inf,inf,inf,0.0,1.0,100,100,1
M/M/S,1.0,1.0,1,1.0,False,inf,inf,inf,inf,0.0,1.0,100,100,1
M/M/S,1.0,1.0,2,0.5,True,1.3333333333333333,0.3333333333333333,1.3333333333333333,0.3333333333333333,0.3333333333333333,0.3333333333333333,100,100,1
M/M/S,1.0,1.0,4,0.25,True,1.0068027210884354,0.006802721088435374,1.0068027210884354,0.006802721088435374,0.3673469387755102,0.02040816326530612,100,100,1
M/M/S,1.0,1.0,8,0.125,True,1.0000014896314204,1.4896314204976263e-06,1.0000014896314204,1.4896314204976263e-06,0.3678793756060938,1.0427419943483384e-05,100,100,1
M/M/1,1.0,5.0,1,0.2,True,0.25,0.05000000000000001,0.25,0.05,0.8,0.2,100,100,5
M/M/S,1.0,5.0,1,0.2,True,0.25,0.05000000000000001,0.25,0.05,0.8,0.2,100,100,5
M/M/S,1.0,5.0,2,0.1,True,0.20202020202020204,0.0020202020202021,0.20202020202020204,0.0020202020202021,0.8181818181818182,0.018181818181818188,100,100,5
M/M/S,1.0,5.0,4,0.05,True,0.2000030239344411,3.023934441101318e-06,0.2000030239344411,3.023934441101318e-06,0.8187302499281814,5.745475438092503e-05,100,100,5

results/mq_randomness.csv (first 25 lines)

model,servers,lambda,mu,rho,arrival_dist,service_dist,n_measured,L,Lq,W,Wq,server_utilization,N,M,R
M/M/1,1,2.0,5.0,0.4,uniform,uniform,95000,0.48071561380403005,0.08057631122023792,0.2404988358922038,0.04031179452568196,0.4001393025837836,100,50,5
M/M/1,1,2.0,5.0,0.4,exponential,exponential,95000,0.6691555701192858,0.2682039345577222,0.33459760146401507,0.1341099098827319,0.40095163556154945,100,50,5

M/M/1,1,2.0,5.0,0.4,poisson,poisson,95000,1.579611527766074,1.181069351984
4948,0.7892736842105263,0.5901368421052632,0.3985421757815792,100,50,5
M/M/2,2,2.0,5.0,0.2,uniform,uniform,95000,0.40435723436903953,0.0037543889
39045711,0.20223979288599359,0.0018777624756259493,0.2003014227149907,100,
50,5
M/M/2,2,2.0,5.0,0.2,exponential,exponential,95000,0.4166570308462085,0.016
919675059696316,0.20857003480050595,0.008469645187186755,0.199868677893253
63,100,50,5
M/M/2,2,2.0,5.0,0.2,poisson,poisson,95000,0.545389594015298,0.146024207783
47483,0.2732,0.07314736842105263,0.19968269311591158,100,50,5
M/M/4,4,2.0,5.0,0.1,uniform,uniform,95000,0.3989797044923245,6.49683988544
4225e-06,0.20019165820594828,3.259847894821034e-06,0.09974330191310418,100
,50,5
M/M/4,4,2.0,5.0,0.1,exponential,exponential,95000,0.39855192498445047,8.13
6140277596436e-05,0.2000076459676375,4.0830069112024304e-05,0.099617640895
42258,100,50,5
M/M/4,4,2.0,5.0,0.1,poisson,poisson,95000,0.40456924273313855,0.0050862775
594274785,0.20262105263157895,0.0025473684210526315,0.09987074129342777,10
0,50,5
M/M/8,8,2.0,5.0,0.05,uniform,uniform,95000,0.39931328623315093,0.0,0.19955
171394707374,0.0,0.04991416077914437,100,50,5
M/M/8,8,2.0,5.0,0.05,exponential,exponential,95000,0.4009131913591124,0.0,
0.1998247705351308,0.0,0.050114148919887996,100,50,5
M/M/8,8,2.0,5.0,0.05,poisson,poisson,95000,0.39550042052144657,0.0,0.198,0
.0,0.04943755256518082,100,50,5
M/M/1,1,2.0,10.0,0.2,uniform,uniform,95000,0.21572948393080169,0.015659707
237266108,0.10792796352691099,0.007834442843648997,0.2000697766935292,100,
50,10
M/M/1,1,2.0,10.0,0.2,exponential,exponential,95000,0.25086670583300935,0.0
5039081998553521,0.12544073950451226,0.025196893713875037,0.20047588584747
583,100,50,10
M/M/1,1,2.0,10.0,0.2,poisson,poisson,95000,0.6181071691757027,0.4171175913
2540266,0.309021052631579,0.20853684210526316,0.20098957785030003,100,50,1
0
M/M/2,2,2.0,10.0,0.1,uniform,uniform,95000,0.2007307086409547,0.0004289857
9701144827,0.1003955726839847,0.0002145574788025873,0.10015086142197023,10
0,50,10
M/M/2,2,2.0,10.0,0.1,exponential,exponential,95000,0.20203949477408747,0.0
021706319467740807,0.10113676800298604,0.0010865731963259494,0.09993443141
365498,100,50,10
M/M/2,2,2.0,10.0,0.1,poisson,poisson,95000,0.23492924577892724,0.036354843
45760003,0.11761052631578947,0.0182,0.0992872011606636,100,50,10
M/M/4,4,2.0,10.0,0.05,uniform,uniform,95000,0.19948726639115366,0.0,0.1000
9419917902096,0.0,0.04987181659778849,100,50,10
M/M/4,4,2.0,10.0,0.05,exponential,exponential,95000,0.19924139695622334,5.
4597730024508315e-06,0.09998614785010744,2.739900838835303e-06,0.049808984
29580427,100,50,10
M/M/4,4,2.0,10.0,0.05,poisson,poisson,95000,0.1998403797282255,0.000504064
0161300485,0.1001578947368421,0.0002526315789473684,0.049834078928023856,1
00,50,10
M/M/8,8,2.0,10.0,0.025,uniform,uniform,95000,0.19965667392429856,0.0,0.099
77585697354002,0.0,0.024957084240536782,100,50,10

```
M/M/8,8,2.0,10.0,0.025,exponential,exponential,95000,0.20045661147697438,0
.0,0.09991238526755758,0.0,0.025057076434623234,100,50,10
M/M/8,8,2.0,10.0,0.025,poisson,poisson,95000,0.19875409143701828,0.0,0.099
07368421052631,0.0,0.024844261429627285,100,50,10
```

Full CSVs (64 + 192 + 135 + 132 rows) accompany this report in `paper3-scheduling/results/`.

Appendix D — Full-resolution plots

Ten figures are embedded inline in the body. The full set is 30 PNGs at 300 dpi:

- `paper3-scheduling/figures/mlfq/` — 24 figures (8 each of heatmaps, 2-D line plots and 3-D surface plots, across the four objectives and two variants).
- `paper3-scheduling/figures/queuing/` — 6 figures (L, Lq, W, Wq vs server rate; L vs offered load; randomness study W / Wq bar comparison).

All figures are regenerated deterministically by `drivers/plot_mlfg.py` and `drivers/plot_queuing.py` from the CSVs in Appendix C.

Appendix E — Sample simulation screenshots

Captured terminal sessions running the same drivers and binaries the body of the paper benchmarks. The runs are intentionally short so the output fits on a single screen; metric format is identical to the full sweep.

```
d Semester/Operating Systems/os-final-project/paper3-scheduling/results/mlfq_A.csv
[10/36] 0.2s
[20/36] 0.3s
[30/36] 0.3s
[36/36] 0.3s

[run_mlfg] done in 0.4s. Optimal configs:
max throughput      = 0.0019 at Q1=2 Q2=8 L1=20 L2=30
min turnaround_mean = 35356.4631 at Q1=2 Q2=24 L1=20 L2=30
min waiting_mean    = 34832.7291 at Q1=2 Q2=24 L1=20 L2=30
min response_mean    = 112.9251 at Q1=2 Q2=48 L1=40 L2=30
```

Screenshot 1: Variant A optimisation output — `run_mlfg.py --variant A` printing the per-config progress lines and the four “Optimal configs” rows at the end (the headline result of §6).

```
[10/108] 0.2s
[20/108] 0.3s
[30/108] 0.4s
[40/108] 0.5s
[50/108] 0.6s
[60/108] 0.6s
[70/108] 0.7s
[80/108] 0.8s
[90/108] 0.8s
[100/108] 0.9s
[108/108] 1.0s

[run_mlfg] done in 1.1s. Optimal configs:
max throughput      = 0.0019 at Q1=2 Q2=8 L1=20 L2=30 T=0
min turnaround_mean = 18678.5091 at Q1=2 Q2=8 L1=20 L2=50 T=100
min waiting_mean    = 18154.7751 at Q1=2 Q2=8 L1=20 L2=50 T=100
min response_mean    = 112.9251 at Q1=2 Q2=48 L1=40 L2=30 T=0
```

Screenshot 2: Variant B optimisation output — same driver with `--variant B`. The optimal configs row now includes the SJF fraction T , demonstrating the §6.3 finding that $T = 100$ minimises mean turnaround.

```
py --simulate --arrivals 10000

[run_queuing] analytic: 135 rows -> results/mq_analytic.csv (0.0s)
[run_queuing] randomness study: 132 rows -> results/mq_randomness.csv (0.9s)
```

Screenshot 3: Queueing analytic + randomness study — `run_queuing.py --simulate` finishing, printing both “analytic: 135 rows” and “randomness study: 132 rows” along with the wall-clock time (≈ 11 s for the full sweep on the Apple M1 Pro).

```
### START Thu May 21 12:39:00 CEST 2026
### MLFQ variant A
[run_mlfq] variant=A configs=64 trials=300 N=500 M=100.0 -> /Users/endribaku/Documents/Epoka/Year 3/Second Semester/Operating Systems/os-final-project/paper3-scheduling/results/mlfq_A.csv
[10/64] 57.5s
[20/64] 124.0s
[30/64] 181.2s
[40/64] 245.2s
[50/64] 303.2s
[60/64] 392.3s
[64/64] 426.4s

[run_mlfq] done in 426.4s. Optimal configs:
max throughput      =      0.0020  at Q1=2 Q2=8 L1=20 L2=30
min turnaround_mean = 168771.3826 at Q1=16 Q2=16 L1=20 L2=30
min waiting_mean    = 168265.6066 at Q1=16 Q2=16 L1=20 L2=30
min response_mean    =   809.1385  at Q1=2 Q2=32 L1=40 L2=30
### MLFQ variant B
[run_mlfq] variant=B configs=192 trials=300 N=500 M=100.0 -> /Users/endribaku/Documents/Epoka/Year 3/Second Semester/Operating Systems/os-final-project/paper3-scheduling/results/mlfq_B.csv
[10/192] 483.6s
[20/192] 1409.5s
```

Screenshot 4: Real-run sweep log — `results/sweep_log.txt`, viewed under `less`. This is the actual progress log saved during the production sweep that produced `mlfq_A.csv` and `mlfq_B.csv`, not a demo run.